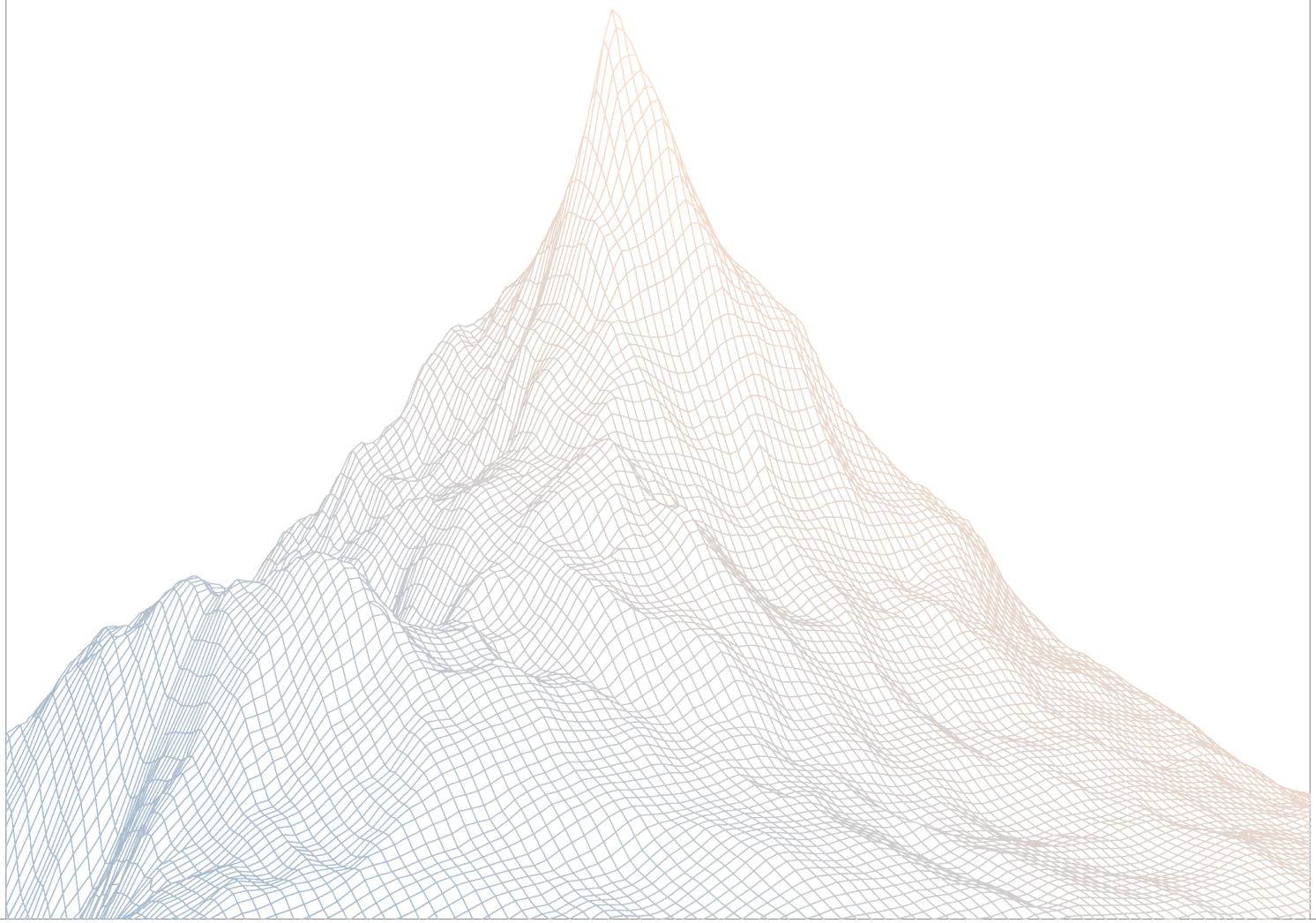
Zenith

Meteora

Smart Contract Security Assessment

VERSION 1.1



AUDIT DATES: August 17th to September 21st, 2026

AUDITED BY: peakbolt
shaflow2

Contents

1 Introduction 2

1.1 About Zenith 3

1.2 Disclaimer 3

1.3 Risk Classification 3

2 Executive Summary 3

2.1 About Meteora 4

2.2 Scope 4

2.3 Audit Timeline 5

2.4 Issues Found 5

3 Findings Summary 5

4 Findings 10

4.1 High Risk 11

4.2 Medium Risk 13

4.3 Low Risk 44

4.4 Informational 97

1 Introduction

1.1 About Zenith

Zenith assembles auditors with proven trAcknowledged records: finding critical vulnerabilities in public audit competitions.

Our audits are carried out by a curated team of the industry's top-performing security researchers, selected for your specific codebase, security needs, and budget.

Learn more about us at https://zenith.security.

1.2 Disclaimer

This report reflects an analysis conducted within a defined scope and time frame, based on provided materials and documentation. It does not encompass all possible vulnerabilities and should not be considered exhaustive.

The review and accompanying report are presented on an "as-is" and "as-available" basis, without any express or implied warranties.

Furthermore, this report neither endorses any specific project or team nor assures the complete security of the project.

1.3 Risk Classification

SEVERITY LEVEL	IMPACT: HIGH	IMPACT: MEDIUM	IMPACT: LOW
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

2

Executive Summary

2.1 About Meteora

Our mission is to build the most secure, sustainable and composable liquidity layer for all of Solana and DeFi.

By using Meteora's DLMM and Dynamic AMM Pools, liquidity providers can earn the best fees and yield on their capital.

This would help transform Solana into the ultimate trading hub for mainstream users in crypto by driving sustainable, long-term liquidity to the platform. Join us at Meteora to shape Solana's future as the go-to destination for all crypto participants.

2.2 Scope

The engagement involved a review of the following targets:

Target	dlmm-pro
Repository	https://github.com/MeteoraAg/dlmm-pro
Commit Hash	`c185afe8b24e9525f36a0445d1e9889a6d3d12c7`
Files	`libs/**/*.rs` `programs/dlmm-pro/src/**/*.rs`

Target	dlmm-pro Mitigation Review
Repository	https://github.com/MeteoraAg/dlmm-pro
Commit Hash	`544cea34ebe133e540c9de352861bc8b7cc90e40`
Files	`libs/**/*.rs` `programs/dlmm-pro/src/**/*.rs`

2.3 Audit Timeline

August 17, 2026	Audit start
September 21, 2026	Audit end
September 20, 2026	Report published

2.4 Issues Found

SEVERITY	COUNT
Critical Risk	0
High Risk	1
Medium Risk	13
Low Risk	26
Informational	18
Total Issues	**58**

3

Findings Summary

ID	Description	Status
H-1	Unchanged-price steps can corrupt the tick-liquidity boundary invariant	Resolved
M-1	Unconditional Q64 price flooring favors Y-to-X limit-order takers	Resolved
M-2	Final swap leg is omitted from dynamic-fee volatility	Resolved
M-3	Malicious LP can increase fees using dust tick boundaries	Resolved
M-4	The limit order may DOS the swap	Resolved
M-5	initialize_pool_with_dynamic_config squares bonding-curve price increases	Resolved
M-6	open_limit_order accepts active-tick asks behind the upward-swap traversal cursor	Resolved
M-7	Relative slippage in open_limit_order can expand by a full tick-spacing interval	Acknowledged
M-8	Filtered small swaps can repeatedly apply the same dynamic-fee decay interval in swap	Acknowledged
M-9	Balance clamping in add_liquidity_imbalance can commit zaps with dust liquidity	Resolved
M-10	Tick-only slippage in add_liquidity_imbalance permits material same-tick quote drift	Resolved
M-11	Market-cap fee schedules use the swap's starting price throughout multi-tick traversal	Acknowledged
M-12	Mutable Token-2022 transfer-fee mints can inflate deposits and confiscate claim settlement value	Resolved
M-13	Later same-side orders can dilute historical limit-order fees after fee decay	Acknowledged

ID	Description	Status
L-1	Swap steps with zero movement after rounding can consume exact input without explicit accounting ownership	Acknowledged
L-2	Relative tick shifts can invalidate preselected tick-array accounts	Resolved
L-3	Prefunded tick-array PDAs can exhaust the instruction trace	Acknowledged
L-4	Frozen reward vaults can permanently block position closure	Acknowledged
L-5	Duplicate tick ranges bypass the stale-index guard	Resolved
L-6	Bonding target validation accepts prices above the highest spacing-aligned tick	Resolved
L-7	Stale bitmap indexes can remove an unintended empty range	Resolved
L-8	Amount-derived liquidity ignores executable tick caps	Acknowledged
L-9	Zero-net transfer-fee swaps can advance pool state without a fill	Acknowledged
L-10	Fresh tick arrays undercount the per-instruction reallocation limit	Resolved
L-11	Uninitialized bitmap extensions block inward liquidity searches	Resolved
L-12	Fully filled limit orders can be blocked by remove-liquidity activation gates	Acknowledged
L-13	BondingCurve activation assumes the target token is token_x and reverses the price milestone when it is token_y	Acknowledged
L-14	Rounding in LP accounting may reduce rewards and leave unaccountable residual dust in the vault.	Acknowledged

ID	Description	Status
L-15	Zero fill fee splitting can credit future limit order makers	Resolved
L-16	Legacy SPL mints bypass freeze-authority validation, allowing a malicious issuer to freeze funded pool reserves or reward vaults	Acknowledged
L-17	Pools created through initialize_launch_pool can omit activation gating	Acknowledged
L-18	The permanent delegate can create liquidity through a reserve self-transfer in add_liquidity	Open
L-19	Bitmap indexes in add_liquidity and remove_liquidity are not bound to expected tick ranges	Resolved
L-20	Valid expanded positions can exceed account limits in refresh_fees_and_rewards	Resolved
L-21	A failure on one token side blocks claiming the other side through claim_fee	Acknowledged
L-22	Vesting refresh in lock_permanent_position can permanently lock more liquidity than intended	Acknowledged
L-23	Orders opened through open_limit_order lAcknowledged a maximum transfer-fee-inclusive debit	Resolved
L-24	SPL multisig owners cannot close limit orders through close_limit_order	Acknowledged
L-25	Stale range indices in expand_position can redirect a subsequent liquidity deposit	Resolved
L-26	Aggregated limit-order fills can leave per-order rounding dust unclaimable in pool reserves	Acknowledged
I-1	get_delta_amount_x_unchecked keeps a narrower intermediate than get_next_sqrt_price_from_amount_in_x_rounding_up	Resolved
I-2	Rent reductions can under-refund historical tick-array rent contributions	Acknowledged

ID	Description	Status
I-3	Primitive safe_shl silently discards overflowing high bits	Resolved
I-4	Pinned ruint version makes wide-integer safe_shl miss overflow	Resolved
I-5	Rent changes break tick-array credit accounting	Acknowledged
I-6	Ceiling-rounded refunds can underfund carried LP rewards	Resolved
I-7	mul_shift_96 places the middle-product carry at the wrong bit position	Resolved
I-8	Dynamic-fee u24 bounds accept 2^24	Resolved
I-9	Expanded arrays fail idempotent initialization	Resolved
I-10	Incorrect and misspelled comments	Resolved
I-11	Q64.64 exponential helper uses non-exact reciprocal and incomplete boundary semantics	Acknowledged
I-12	Writable pool accounts unnecessarily serialize otherwise-independent claims	Resolved
I-13	Generic tick-array insertion bypasses the TickOps reference predicate	Resolved
I-14	Several validation paths return misleading existing PoolError variants	Resolved
I-15	Protocol fee claims can fail for Token-2022 accounts requiring incoming transfer memos	Acknowledged
I-16	Unreachable schedule endpoints in lock_vesting_position can permanently lock liquidity	Resolved

ID	Description	Status
I-17	Partial limit-order fills in swap can strand uncredited input	Acknowledged
I-18	withdraw_ineligible_rewards allows the reward authority to redirect unused campaign tokens	Acknowledged

4

Findings

4.1 High Risk

A total of 1 high risk findings were identified.

[H-1] Unchanged-price steps can corrupt the tick-liquidity boundary invariant

SEVERITY: High	IMPACT: High
STATUS: Resolved	LIKELIHOOD: Medium

Target:

- ix_swap.rs#413

Description:

`swap_with_tick` advances the swap price while preserving which side of a tick boundary contains the active liquidity.

However, a non-target step recomputes `current_tick_id` from `next_sqrt_price` even when integer rounding leaves the square-root price unchanged.

At an already-crossed boundary, this changes the stored tick without applying `liquidity_net` or updating outside growth, potentially causing incorrect swap payouts, misallocated fees or rewards, and reverting swaps or withdrawals.

Example Scenario:

1. A spacing-100 position spans ticks `-100100` to `-100000`; after crossing downward, the price is exactly at tick `-100000`, `tick_current_id` is `-100001`, and lower-side liquidity is active.
2. Let the boundary square-root price be `S = 124324258982887573` and active liquidity be `L = S * (S - 1) + 1 = 154565213716441012630353901749427 57`, which remains below the spacing-100 gross-liquidity limit.
3. A downward swap supplies one raw token-X unit, and upward division rounds `ceil(L * S / (L + S))` bAcknowledged to S, so input is consumed without changing the integer square-root price.

4. Because `s` differs from the target tick price, the non-target branch maps it to tick `-100000` without processing a crossing.
5. The pool persists `tick_current_id = -100000` while its liquidity and outside-growth accounting still represent tick `-100001`, corrupting the boundary-side invariant.

```
if swap_step_result.next_sqrt_price ≠ tick.base().sqrt_price {
 *current_sqrt_price = swap_step_result.next_sqrt_price;
 *current_tick_id = tick_id_from_sqrt_price(swap_step_result
 .next_sqrt_price)?;
}
```

Recommendations:

Recompute `current_tick_id` only when the square-root price actually changes, preserving the stored boundary side for zero-movement steps.

```
let next_sqrt_price = swap_step_result.next_sqrt_price;
if next_sqrt_price ≠ *current_sqrt_price {
 *current_tick_id = tick_id_from_sqrt_price(next_sqrt_price)?;
}
*current_sqrt_price = next_sqrt_price;
```

Meteora: Resolved with PR-108.

Zenith: Verified.

4.2 Medium Risk

A total of 13 medium risk findings were identified.

[M-1] Unconditional Q64 price flooring favors Y-to-X limit-order takers

SEVERITY: Medium	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target:

- tick.rs#375
- tick_math.rs#66

Description:

`get_amount_in` and `get_amount_out` convert a tick's square-root price into a Q64 execution price for limit-order fills.

However, `get_price_from_sqrt_price` always rounds the intermediate price down. Downward price rounding is conservative for X-to-Y swaps, but Y-to-X swaps require the price to round up so required Y input is not understated and X output is not overstated.

A Y-to-X limit-order taker can consequently pay one raw Y unit less than required, while some valid partial fills near the minimum price can revert because division by the underestimated price produces an overflowing output.

Example Scenario:

1. At tick `8,882`, `sqrt_price = 28,759,428,992,292,043,991` and the exact price is approximately `2.43064242043096666668558` Y per X.
2. A Y-to-X taker requests exactly `30,000,000,000,000` raw X units, which is `30,000 X` when X has 9 decimals.
3. Applying the exact price requires `72,919,272,612,929.000000567 ...` raw Y units, so conservative final rounding must charge `72,919,272,612,930` raw Y units.
4. The current floored intermediate price instead calculates `72,919,272,612,928.999999465 ...` raw Y units and rounds this to `72,919,272,612,929`.

5. Rounding the intermediate price upward calculates `72,919,272,612,929.000001091` ... and charges `72,919,272,612,930`, preventing the one-raw-unit undercharge.

```
let price = get_price_from_sqrt_price(sqrt_price);
let prod = U256::from(price).safe_mul(U256::from(out_amount))?;
let amount = prod.div_ceil(U256::from(Q64));
```

Recommendations:

Allow `get_price_from_sqrt_price` to round in either direction and use a rounded-up intermediate price for Y-to-X conversions while retaining the existing final input-up and output-down rounding.

```
pub fn get_price_from_sqrt_price(sqrt_price: u128,
 round_up: bool) → Result<u128> {
 let mut product = U256::from(sqrt_price)
 .safe_mul(U256::from(sqrt_price))?;
 if round_up {
 product = product.safe_add(U256::from(Q64 - 1))?;
 }
 try_cast(product >> 64)
}

let price = get_price_from_sqrt_price(sqrt_price, !swap_for_y)?;
```

Meteora: Resolved with PR-115.

Zenith: Verified.

[M-2] Final swap leg is omitted from dynamic-fee volatility

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target:

- mod.rs#273

Description:

`update_post_swap` finalizes the variable-fee state after a swap changes the pool price.

However, it updates only `last_update_timestamp` and never checkpoints the final `sqrt_price` into `volatility_accumulator`, while the swap loop updates the accumulator only before each price-changing step.

The final step's price movement is permanently omitted when the next swap occurs after `filter_period`, causing the dynamic fee to underestimate volatility and reducing fees collected for liquidity providers and the protocol.

This differs from standard DLMM's discrete-bin swap flow. DLMM updates the volatility accumulator using the current `active_id` before executing against every nonempty bin and advances to the next bin only when input remains. Each DLMM bin has a fixed price, so when a swap finishes in its final bin, that ending price has already been included in the pre-fill accumulator update and there is no intra-bin final price movement to checkpoint.

DLMM Pro instead has continuous liquidity between ticks. Its final step can stop at a new `sqrt_price` inside a tick range after volatility was updated using the step's starting price. Because `update_post_swap` records only the timestamp, this final continuous movement is never included and can be erased when the next swap reanchors `sqrt_price_reference` after `filter_period`.

DAMM-v2 also supports continuous price movement and explicitly calls `update_volatility_accumulator(self.sqrt_price)` during `update_post_swap` before updating `last_update_timestamp`. DLMM Pro retains the timestamp update from that lifecycle but omits the final accumulator checkpoint.

Example Scenario:

1. A pool starts with `sqrt_price_reference = 1_000_000` and `volatility_accumulator = 0`.

2. A one-step swap moves `sqrt_price` to `1_000_500`, corresponding to a ten-tick delta.
3. `update_post_swap` advances the timestamp but leaves `volatility_accumulator = 0` instead of `100_000`.
4. The next swap arrives after `filter_period`, reanchors `sqrt_price_reference` to `1_000_500`, and can no longer reconstruct the omitted movement.

```
let delta_tick = VariableFee::get_delta_tick(start_sqrt_price,
 self.sqrt_price);
if delta_tick > MIN_DELTA_TICK_FOR_DYNAMIC_FEE_UPDATE {
 self.variable_fee.last_update_timestamp = current_timestamp;
}
```

Recommendations:

Checkpoint the final pool price into the volatility accumulator before updating `last_update_timestamp`, guarded by `is_dynamic_fee_enable()`.

```
if self.fee_config.dynamic_fee.is_dynamic_fee_enable() {
 self.variable_fee.update_volatility_accumulator(
 self.sqrt_price,
 &self.fee_config.dynamic_fee,
 );
}
```

Meteora: Resolved with PR-167.

Zenith: Verified.

[M-3] Malicious LP can increase fees using dust tick boundaries

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target:

- ix_swap.rs#694
- mod.rs#266

Description:

`swap` refreshes the volatility accumulator before each non-empty initialized tick and then applies the resulting dynamic fee while swapping through that tick. Empty initialized ticks are skipped, and the accumulator is not refreshed from the final swap price until a later swap.

Consequently, a malicious LP can increase the dynamic fees earned by its meaningful liquidity position using comparatively cheap dust positions. The LP first provides substantial liquidity across a wide range, then overlays that range with many negligible positions whose endpoints create non-empty intermediate ticks. Each endpoint splits the wide range into another swap step and refreshes the volatility accumulator before the swap continues.

As price moves across the range, the resulting segments are charged using progressively refreshed, potentially higher dynamic fees. Because the malicious LP's wide position remains active throughout the range and supplies nearly all of its liquidity, it receives nearly all of the increased market-making fees. The dust positions are needed primarily to create fee checkpoints, rather than to provide economically meaningful liquidity.

Classic DLMM has a superficially similar per-bin update, but it does not have the same behavior. DLMM liquidity executes at discrete fixed-price bins, and the accumulator at each non-empty bin is overwritten from the stored reference and that bin's `active_id`. Adding negligible liquidity to an intermediate bin therefore creates a fee checkpoint only for the newly added liquidity; it does not change the accumulator later calculated for an existing bin. In DLMM Pro, by contrast, an initialized tick is a boundary within continuous concentrated liquidity, so inserting a boundary changes the fee applied to the pre-existing liquidity after that boundary.

This also differs from DAMM-v2, which uses one stored volatility-based fee throughout a swap and updates the accumulator from the final price afterward.

```
if !tick.base().is_non_zero_liquidity() {
 return Ok(false);
}

pool.base_mut().update_pre_swap_tick(current_sqrt_price);
let result = swap_with_tick(...)?;
```

Example Scenario

1. A malicious LP supplies substantial liquidity across ticks `0` through `1_000`. With no internal initialized boundaries, a swap across that liquidity uses the fee calculated at the initial checkpoint.
2. The same LP opens negligible positions throughout that range, for example `[0, 100]`, `[100, 200]`, and so on. Their endpoints make ticks `100`, `200`, and the remaining intermediate ticks non-empty while adding almost no liquidity relative to the wide position.
3. A swap moving across the range now refreshes the volatility accumulator at every dust-created boundary. Later portions of the swap are charged progressively updated, potentially higher dynamic fees.
4. The wide position remains active across every segment and owns nearly all active liquidity, so the malicious LP captures nearly all of the additional market-making fees caused by its dust boundaries.
5. If sufficiently many boundaries are added, the additional tick processing can also increase compute consumption or make a long-range swap exceed its compute budget.

Recommendations:

Make fee evolution independent of initialized-tick topology. One option is to follow DAMM-v2's ordering: use one stored volatility-based fee throughout the swap, remove the per-tick accumulator refresh, and update the accumulator from the final square-root price in `update_post_swap` so the completed movement affects subsequent swaps.

```
pub fn update_post_swap(
 &mut self,
 start_sqrt_price: u128,
 current_timestamp: u64,
) -> Result<()> {
 if self.fee_config.dynamic_fee.is_dynamic_fee_enable() {
 self.variable_fee.update_volatility_accumulator(
 self.sqrt_price,
 &self.fee_config.dynamic_fee,
```

```
 );
 }

 let delta_tick = VariableFee::get_delta_tick(start_sqrt_price,
 self.sqrt_price);
 if delta_tick > MIN_DELTA_TICK_FOR_DYNAMIC_FEE_UPDATE {
 self.variable_fee.last_update_timestamp = current_timestamp;
 }
 Ok(())
}
```

Meteora: Resolved with PR-167.

Zenith: Verified.

[M-4] The limit order may DOS the swap

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target:

- tick.rs
- ix_open_limit_order.rs

Description:

Limit orders are stored as a token amount at a chosen tick. During an exact-input swap, when the swap reaches a tick with compatible limit-order liquidity, the program first calculates how much input would be required to fill the entire outstanding order:

```
let total_amount_in =
 Tick::get_amount_in(self.sqrt_price, total_out_amount, swap_for_y,
 Rounding::Up)?;

if amount_in ⩾ total_amount_in {
 // full fill
} else {
 let out_amount = Tick::get_amount_out(amount_in, self.sqrt_price,
 swap_for_y)?;
 // partial fill
}
```

This full-fill quote is cast bAcknowledged to `u64` inside `Tick::get_amount_in(...)`. At high enough price ticks, a valid limit order can have a full-fill counter-token amount greater than `u64::MAX`, even though the current swap input is small enough to partially fill the order safely. In that case, `get_amount_in(...)` fails before the code reaches the partial-fill branch, causing the whole swap to revert.

The attAcknowledgeder does not need to provide the overflowing counter-token amount. They only need to place the limit order amount on the output side. For example, with an ask order where `X = wSOL` and `Y = meme`:

- `0.001 SOL` (`1,000,000` lamports) overflows around tick `305,475`, which corresponds to roughly `1.84e13 Y_base / X_base`.
- `1000 SOL` (`1,000,000,000,000` lamports) overflows around tick `167,313`; with

`tick_spacing = 256`, the usable aligned tick is about `167,424`, or roughly `18.65 million Y_base / X_base`.

The first case is cheap but requires a very high price region. The second case requires large capital, but the price range is realistic for some wSOL / low-price meme pools. The mirrored bid-side case exists at negative ticks.

As a result, a malicious limit order can make swaps that reach the affected tick revert instead of partially filling the order. This can block routing through that price level until the order is cancelled or the price path avoids the tick.

Recommendations:

It is recommended to check the tick state of the order price and call `get_amount_in` when enabling a limit order. If an overflow occurs, prevent the limit order from being enabled.

Meteora: Resolved with PR-110.

Zenith: Verified.

[M-5] `initialize_pool_with_dynamic_config` squares bonding-curve price increases

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target:

- activation_bonding_curve.rs#41

Description:

`PodAlignedActivationBondingCurve` locks liquidity changes until the token price reaches the configured increase milestone.

However, it applies `target_increase_bps` directly to the square-root price even though token price is proportional to the square of that value.

The effective price increase is squared, so liquidity can remain locked until a substantially higher price than the configured milestone.

Example Scenario:

1. A creator configures `target_increase_bps` as `20_000` to unlock liquidity when price doubles from 1 to 2.
2. The program multiplies the initial square-root price by 2.
3. The corresponding token price rises from 1 to 4, so the target flag remains unset at price 2.
4. Non-whitelisted users cannot add or remove liquidity until price reaches 4.

```
let target_sqrt_price: u128 = try_cast(
 U192::from(init_sqrt_price)
 .safe_mul(U192::from(target_increase_bps))?
 .safe_div(U192::from(MAX_BASIS_POINT))?,
)?;
```

Recommendations:

Apply the fixed-point square root of the requested price multiplier to `init_sqrt_price`, or rename the field and document that it is a square-root-price multiplier.

Meteora: Resolved with @5706fab00e...

Zenith: Verified.

[M-6] `open_limit_order` accepts active-tick asks behind the upward-swap traversal cursor

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target:

- ix_open_limit_order.rs#90

Description:

`open_limit_order` permits an ask order when its lowest tick equals the pool's current active tick:

```
amount_in_ticks[0].id ⩾ active_id
```

However, the swap logic treats the active tick as already entered. When price moves upward, it only processes ticks strictly greater than the current tick:

```
tick_current_id < tick_id
```

Consequently, an ask placed at `tick_id = active_id` is accepted and funded but is behind the upward-swap traversal cursor. Upward swaps skip the order even as price moves above its execution tick.

The order becomes reachable only if price first crosses below that tick and subsequently moves upward again. Until the maker cancels it or that price path occurs, the pending ask also prevents bid-side orders from being placed at the same tick.

Example Scenario

1. The pool's current tick is `100`.
2. A maker opens an ask for `1,000` token X at tick `100`.
3. Validation accepts the order because `100 ⩾ 100`.
4. A Y-to-X swap moves price upward.
5. The swap only processes ticks satisfying `tick_id > 100`, so it skips the ask at tick `100`.

6. The order remains unfilled after price moves above `100`.
7. A bid at tick `100` is rejected because that tick still contains a pending ask.

Impact

Makers can have funds unexpectedly stranded in orders that do not execute when price moves in the intended direction. Pending asks can also temporarily prevent opposite-side orders at the affected tick.

Recommendations:

Require ask ticks to be strictly above the active tick so placement matches upward traversal:

```
require!(
 amount_in_ticks[0].id > active_id,
 PoolError::InvalidParameters
);
```

Add regression tests confirming that:

- an ask at the active tick is rejected;
- an ask at the next usable tick is accepted;
- bids at or below the active tick remain accepted.

Meteora: Resolved with @5f5a8db7f4...

Zenith: Verified.

[M-7] Relative slippage in `open_limit_order` can expand by a full tick-spacing interval

SEVERITY: Medium	IMPACT: Medium
STATUS: Acknowledged	LIKELIHOOD: Medium

Target:

- ix_open_limit_order.rs#62

Description:

`open_limit_order` checks a relative order's tolerated movement against the observed and current active tick IDs.

However, it validates the raw tick difference before flooring the current active ID to `tick_spacing`, so crossing one alignment boundary can move the resolved limit by an entire spacing interval.

The order can execute at a materially worse limit price than the raw slippage bound authorized by the maker.

Example Scenario:

1. Tick spacing is `256`, the observed active ID is `255`, the current active ID is `256`, and `max_slippage` is `1`.
2. The raw difference is `1`, so validation succeeds.
3. A relative bid with offset `0` resolves from base tick `0` when observed but from base tick `256` at execution.
4. The maker's limit moves by `256` ticks, approximately `2.59%`, despite authorizing one tick of movement.

```
let tick_shift = offchain_active_id.abs_diff(active_id);
require!(tick_shift ≤ max_slippage, PoolError::ExceededSlippage);

let tick_spacing_aligned_current_id =
 align_id_to_spacing(active_id, tick_spacing)?;
```

Recommendations:

Compare the spacing-aligned observed and current bases, or preserve the exact ticks derived from the observed active ID.

```
let observed_base = align_id_to_spacing(offchain_active_id, tick_spacing)?;
let current_base = align_id_to_spacing(active_id, tick_spacing)?;
require!(observed_base.abs_diff(current_base) ⩽ max_slippage,
 PoolError::ExceededSlippage);
```

Meteora: This behavior was observed after modifying the slippage logic in @8315ddc1b2... and @362741ad99...

Zenith: Verified.

[M-8] Filtered small swaps can repeatedly apply the same dynamic-fee decay interval in `swap`

SEVERITY: Medium	IMPACT: Medium
STATUS: Acknowledged	LIKELIHOOD: Medium

Target:

- mod.rs#625

Description:

The `MIN_DELTA_TICK_FOR_DYNAMIC_FEE_UPDATE` threshold intentionally prevents small swaps from refreshing the timestamp of the last meaningful price movement. This ensures that dust trades cannot keep dynamic fees elevated.

However, `update_references` also uses that same timestamp to determine whether volatility decay should be applied:

```
let elapsed = current_timestamp.saturating_sub(self.last_update_timestamp);

if elapsed ⩾ dynamic_fee_config.filter_period.into() {
 self.sqrt_price_reference = sqrt_price_current;
 // Reduce or reset volatility_reference ...
}
```

When a swap remains at or below the threshold, `update_post_swap` deliberately does not update `last_update_timestamp`:

```
if delta_tick > MIN_DELTA_TICK_FOR_DYNAMIC_FEE_UPDATE {
 self.variable_fee.last_update_timestamp = current_timestamp;
}
```

Consequently, the next small swap still observes the same elapsed interval as eligible for decay. It can therefore reduce the volatility reference again and reset the price reference, even when no additional decay period has passed.

The issue is not that small swaps are filtered. The issue is that one timestamp serves two separate purposes:

1. Recording the last meaningful price movement.
2. Recording when time-based decay was last applied.

Example Scenario

Assume:

- `volatility_accumulator = 1,000,000`
- `reduction_factor = 5,000` (50%)
- The filter period has elapsed, but the decay period has not.
- Every swap moves the price by no more than the four-tick threshold.

The first small swap reduces the effective accumulator to approximately `500,000`. Because the swap is filtered, `last_update_timestamp` remains unchanged.

Additional small swaps can repeatedly apply the reduction:

```
1,000,000 -> 500,000 -> 250,000 -> 125,000 -> ... -> 976
```

Ten reductions leave the accumulator near `976`. Because the variable-fee component is quadratic in the accumulator, it becomes approximately one-millionth of its original value.

Resetting `sqrt_price_reference` on every small swap also prevents their cumulative price movement from being reflected in the volatility calculation.

Impact

A trader can split cumulative price movement across multiple swaps below the threshold and reduce the dynamic-fee surcharge much faster than intended.

This lowers fees collected by liquidity providers and the protocol and weakens the pool's protection against volatile trading.

Recommendations:

Keep the existing threshold for identifying meaningful price movement, but trAcknowledged decay application separately.

For example, introduce a dedicated reference-update timestamp:

```
self.last_reference_update_timestamp = current_timestamp;
```

Use this timestamp to determine whether the filter or decay interval has elapsed. Continue

using `last_update_timestamp` only to trAcknowledged swaps that exceed the small-trade threshold.

This preserves the intended filtering of small swaps while ensuring that one elapsed interval cannot apply decay repeatedly.

Meteora: Acknowledged with comment in attached commit @9682610533....

Zenith: Acknowledged by client.

[M-9] Balance clamping in `add_liquidity_imbalance` can commit zaps with dust liquidity

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target:

- ix_add_liquidity_imbalance.rs#94

Description:

`build_handle_add_liquidity_params` derives position liquidity from caller-specified token maxima and the source accounts' live balances.

However, each maximum is silently clamped to the available balance and the resulting liquidity is accepted as long as it is nonzero, with no caller-provided minimum liquidity or token-utilization bound.

A stale quote or underperforming preceding swap can let an atomic zap or rebalance succeed while deploying only dust liquidity, committing fees and other transaction legs without achieving the caller's intended position size.

Example Scenario:

1. A user signs a rebalance expecting to deposit up to `50,000` X and `50,000` Y after a preceding swap.
2. The swap produces only `1` Y because its execution deteriorates before landing.
3. The wrapper clamps Y to `1`, derives a positive dust amount of liquidity, and passes the common handler's nonzero check.
4. The entire transaction succeeds even though almost none of the intended liquidity is added.

```
let transfer_fee_included_amount = max_amount.min(available_token_amount);

let liquidity = liquidity_x.min(liquidity_y);
let liquidity: u128 = try_cast(liquidity)?;
```

Recommendations:

Add a caller-provided `min_liquidity` parameter and reject the instruction when the derived liquidity is below it; zap integrations may additionally require minimum X and Y amounts used.

```
require!(liquidity ⩾ self.min_liquidity, PoolError::ExceededSlippage);
```

Meteora: Resolved with PR-113.

Zenith: Verified.

[M-10] Tick-only slippage in `add_liquidity_imbalance` permits material same-tick quote drift

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target:

- ix_add_liquidity_imbalance.rs#75

Description:

`AddLiquidityImbalanceParameters::validate` limits price movement between the caller's observation and execution before deriving the deposit composition.

However, it compares only integer tick IDs even though liquidity and token requirements use the continuous square-root price, so material movement within one tick is unrestricted even when `max_tick_slippage` is zero.

A narrow position can receive drastically less liquidity and a different token composition than quoted while the instruction's slippage check still succeeds.

Example Scenario:

1. A caller quotes a one-tick range from tick `0` to tick `1` at its lower boundary using maxima of `100,000` X and `1,000` Y.
2. The caller sets `observed_tick_id` to `0` and `max_tick_slippage` to `0`.
3. Before execution, the square-root price moves to one unit below the tick `1` boundary while `tick_current_id` remains `0`.
4. The guard passes, but the binding token changes and the derived liquidity can fall to roughly `1%` of the quoted amount.

```
let tick_delta = current_tick_id.abs_diff(self.observed_tick_id);
require!(tick_delta ≤ self.max_tick_slippage,
 PoolError::InvalidParameters);

let lower_sqrt_price = position_lower_sqrt_price.max(current_sqrt_price);
```

Recommendations:

Add caller-specified lower and upper square-root-price bounds and enforce them before deriving liquidity, together with a minimum-liquidity bound.

```
require!(
 current_sqrt_price ≥ self.min_sqrt_price
 && current_sqrt_price ≤ self.max_sqrt_price,
 PoolError::ExceededSlippage
);
```

Meteora: Resolved with PR-113.

Zenith: Verified.

[M-11] Market-cap fee schedules use the swap's starting price throughout multi-tick traversal

SEVERITY: Medium	IMPACT: Medium
STATUS: Acknowledged	LIKELIHOOD: Medium

Target:

- pool/mod.rs
- tick.rs
- ix_swap.rs
- market_cap_scheduler_base_fee.rs

Description:

While a market-cap fee schedule is active, the linear and exponential schedulers derive the applicable fee period from the pool's current square-root price. Prices at or below `init_sqrt_price` use period zero, and each configured `sqrt_price_step_bps` increase advances the period. Both schedulers reduce the base fee as the period increases, so a downward swap should progressively move from lower-fee periods into higher-fee periods as its execution price falls.

The swap traversal initializes a local `current_sqrt_price` from the stored pool price and updates that local value after each processed tick. The final value is not written bAcknowledged to `pool.sqrt_price` until the traversal has completed:

```
let mut current_sqrt_price = pool.base().sqrt_price;

// ... process ticks and update current_sqrt_price ...

pool.base_mut().sqrt_price = current_sqrt_price;
```

Deferring the pool-state write is safe as long as calculations performed during the traversal use the local price. `swap_with_tick( ... )` does pass the local `current_sqrt_price` into `get_swap_result_exact_in( ... )`, and that value is used for the swap amount calculation. However, the tick obtains its trading fee without forwarding the same price:

```
pub fn get_swap_result_exact_in(
 &self,
 current_point: u64,
```

```
 input_amount: u64,
 fee_setting: &FeeSetting,
 swap_for_y: bool,
 current_sqrt_price: u128,
 current_liquidity: u128,
 pool: &Pool,
) -> Result<SwapStepResult> {
 let mut actual_trading_fee = 0;
 let trading_fee_numerator =
 pool.get_total_trading_fee_numerator(current_point)?;

 // ...

 let filled_amount_result = self.get_filled_amount_result_exact_in(
 excluded_fee_input_amount,
 swap_for_y,
 current_sqrt_price,
 current_liquidity,
 )?;
```

`Pool::get_total_trading_fee_numerator(...)` instead supplies the stored `self.sqrt_price` to the base-fee scheduler:

```
pub fn get_total_trading_fee_numerator(&self,
 current_point: u64) -> Result<u64> {
 self.fee_config.get_total_trading_fee_numerator(
 current_point,
 self.activation_point,
 self.init_sqrt_price,
 self.sqrt_price,
 &self.variable_fee,
 )
}
```

During a swap, the program therefore holds two different price values:

- `current_sqrt_price` is the local execution price. It changes after each processed tick and is correctly used to calculate the exchanged token amounts.
- `pool.sqrt_price` remains equal to the price at the beginning of the instruction until the entire swap finishes. It is incorrectly used to select the market-cap base-fee period.

For example, consider an illustrative linear schedule in which the base fee is `2%` above a square-root price of `130`, `3%` from `120` to `129`, `4%` from `110` to `119`, and `5%` below `110`. A downward swap starts at `135` and processes tick steps beginning at `135`, `128`, `118`, and `108`. The intended and actual rates are:

Tick-step starting price	Intended fee	Actual fee
135	2%	2%
128	3%	2%
118	4%	2%
108	5%	2%

The correct local price reaches every new period, but the fee lookup continues to see the original stored price of `135`. If each step had an equal taxable amount of `100` units, this simplified example would charge `8` units instead of `14`. The real calculation uses the protocol's included/excluded-fee formulas and rounding, but the result is the same: the trader pays less and all fee recipients accrue less.

The dynamic-fee component is not stale in the same way because `update_pre_swap_tick( ... )` receives the local price before each tick. This issue specifically affects the price-dependent market-cap base-fee component.

Recommendations:

Pass the local `current_sqrt_price` into `get_total_trading_fee_numerator( ... )` and use it instead of `self.sqrt_price`. Additionally, if a tick step may cross a fee-period boundary, stop at that boundary and recalculate the fee rate.

Meteora: Acknowledged, We choose the current design because that help base fee is more deterministic at time client calculate quoting.

Zenith: Acknowledged by Client.

[M-12] Mutable Token-2022 transfer-fee mints can inflate deposits and confiscate claim settlement value

SEVERITY: Medium	IMPACT: Medium
STATUS: Resolved	LIKELIHOOD: Medium

Target:

- token.rs
- ix_claim_protocol_fee.rs
- ix_claim_partner_fee.rs
- ix_open_limit_order.rs
- ix_claim_fee.rs
- ix_claim_reward.rs
- ix_cancel_limit_order.rs
- ix_withdraw_ineligible_rewards.rs

Description:

Token-2022 mints with `TransferFeeConfig` are treated as permissionlessly supported mints. The mint validation logic allows the extension without checking whether the transfer-fee configuration authority or withheld-fee withdrawal authority has been revoked:

```
match e {
 ExtensionType::TransferFeeConfig
 | ExtensionType::MetadataPointer
 | ExtensionType::TokenMetadata => {
 // permissionless supported
 }
 // ...
}
```

This means an externally controlled Token-2022 mint can be admitted while its issuer still controls future transfer-fee settings. The protocol correctly uses the current epoch's transfer fee when performing transfers, but order-entry and payout flows do not let the affected user enforce a maximum transfer-fee rate, maximum gross debit, or minimum net received amount.

For open limit orders, the user chooses net order amounts in `ticks`. The instruction has optional tick-position slippage through `relative_tick.max_slippage`, but it does not

include a token-transfer-fee bound. After writing the order into tick state, it computes the gross amount required by the mint's current transfer-fee configuration and debits that amount from the user:

```
let transfer_fee_included_amount =
 calculate_transfer_fee_included_amount(&ctx.accounts.token_mint,
 total_amount)?.amount;

transfer_from_user(
 // ...
 transfer_fee_included_amount,
 // ...
)?;
```

If the mint authority raises the transfer fee between user signing and execution, the transaction can still succeed while charging a much larger gross amount than the user expected for the same net limit-order deposit. The existing tick slippage check only protects where the order is placed; it does not cap how many tokens are pulled from the owner.

Several payout paths settle protocol accounting using the gross claimable amount and then call `transfer_from_pool( ... )` with that same gross amount. For example, position fee claims reset the pending fee fields before the outbound transfer is executed:

```
pending_fee_x = pending_fee_x.safe_add(position.base()
 .fee_pending_x.into())?;
pending_fee_y = pending_fee_y.safe_add(position.base()
 .fee_pending_y.into())?;
position.base_mut().reset_pending_fee();

transfer_from_pool(
 // ...
 pending_fee_x,
 Some(memo_transfer_ctx),
 parsed_transfer_hook_accounts.transfer_hook_x,
)?;
```

The same pattern exists across position reward claims, limit-order cancellation proceeds, partner fee claims, protocol fee claims, and ineligible reward withdrawal. In the limit-order path, the tick and limit-order records are cancelled first, and the resulting gross owner amounts are later transferred without a net-received bound:

```
let result = tick
 .base()
 .get_cancel_limit_order_result(lo_tick, collect_fee_mode)?;
tick.base_mut().apply_cancel_limit_order(lo_tick, &result)?;
```

```
final_result.apply_cancel_limit_order(&result)?;

transfer_from_pool(
 // ...
 owner_amount_x,
 Some(memo_transfer_ctx),
 parsed_transfer_hook_accounts.transfer_hook_x,
)?;
```

An external mint controller can also wait until fees, rewards, or limit-order proceeds have accumulated, schedule a high Token-2022 transfer fee for a future epoch, and let an authorized claim or cancellation settle after the fee becomes active. The token transfer succeeds, the protocol accounting is settled by the gross amount, but the receiver only gets the net amount after the mint-level transfer fee is withheld. If the same controller also has the withheld-fee withdrawal authority, it can later collect the withheld tokens.

For example, a user may sign an open limit order expecting to deposit `1,000,000` net units while the mint charges zero transfer fee. If the mint's external fee authority schedules `9,999` bps with a high maximum fee before execution, the program can compute and pull a far larger gross amount to make the reserve receive the same net order amount. The same fee change can later affect claims: a position, partner, or limit-order owner with `1,000,000` gross units claimable may receive only about `100` net units while roughly `999,900` units are withheld by the mint. In both cases, the transfer succeeds, so transaction atomicity does not restore the user's expected price bound.

Recommendations:

Reject mutable `TransferFeeConfig` mints unless explicitly approved by `TokenBadge`, or add user-provided `max_gross_amount`, `min_net_amount`, and `max_transfer_fee_bps` checks to every order-entry, claim, cancellation, and withdrawal path before moving tokens or settling accounting.

Meteora: Resolved with PR-127.

Zenith: Resolved. Adds user-specified maximum gross deposits and minimum net payouts, reverting transactions when Token-2022 transfer fees would breach those bounds.

[M-13] Later same-side orders can dilute historical limit-order fees after fee decay

SEVERITY: Medium	IMPACT: Medium
STATUS: Acknowledged	LIKELIHOOD: Medium

Target:

- tick.rs
- tick.rs
- tick.rs
- limit_order.rs
- pool/mod.rs

Description:

Limit orders at the same tick and side share one tick-level fee bucket. The tick stores aggregate fulfilled amounts and aggregate limit-order fees:

```
pub fulfilled_order_amount_x: u64;
pub fulfilled_order_amount_y: u64;
pub limit_order_fee_ask_side: u64;
pub limit_order_fee_bid_side: u64;
```

However, each `LimitOrderData` only stores `amount`, `age`, `tick_id`, and side. It does not store a fee-growth checkpoint for the fees that had already accrued before the order was opened.

When a new same-side order is placed, `place_limit_order(...)` only checks that it matches the current tick side and then adds the amount to `open_order_amount`. Later swaps merge all processed and open-order fills into the same side-level accumulators:

```
let fulfilled_order_amount = swap_result
 .filled_amount_result
 .get_fulfilled_order_amount()?;

self.fulfilled_order_amount_x = self
 .fulfilled_order_amount_x
 .safe_add(fulfilled_order_amount)?;
```

```
self.limit_order_fee_ask_side = self
 .limit_order_fee_ask_side
 .safe_add(swap_result.limit_order_fee)?;
```

On cancellation, the order receives fees by its share of the current global fulfilled amount:

```
let (limit_order_fee, _) = calculate_shared_amounts(
 self.limit_order_fee_ask_side,
 fulfilled_amount,
 self.fulfilled_order_amount_x,
 Rounding::Down,
)?;
```

This is unsafe when fee density changes over time. The protocol supports time-scheduled fees, market-cap scheduled fees, and dynamic volatility fees. `get_total_trading_fee_numerator(...)` combines the base fee and dynamic fee for each swap, capped by `MAX_FEE_NUMERATOR`. Therefore, the same tick can contain fills from a high-fee period and a later low-fee period.

Example:

1. A victim ask order at tick `T` is filled for `1,000` units during a high-fee period. The fill produces `495` maker fee units after the 50% limit-order protocol fee split.
2. The victim has not canceled yet, so the tick still holds `fulfilled_order_amount_x = 1,000` and `limit_order_fee_ask_side = 495`.
3. The fee later decays. An attAcknowledgeder places a same-side ask order for `99,000` units at the same tick.
4. A later low-fee swap fills the attAcknowledgeder order and adds only `5` maker fee units.

The tick now has:

```
fulfilled_order_amount_x = 100,000
limit_order_fee_ask_side = 500
```

Cancellation distributes the whole fee bucket by current fulfilled share:

```
victim receives floor(500 * 1,000 / 100,000) = 5
attAcknowledgeder receives about 495
```

The attAcknowledgeder receives most of the high-fee maker fees earned before the attAcknowledgeder order existed. This does not make the pool insolvent, but it transfers limit-order fee ownership between makers.

Recommendations:

As long as the same side of the tick still has unsettled `processed` / `fulfilled` amount or fees, do not allow new same-side orders to be added.

Meteora: Acknowledged, added in attached commit to confirm intended design @b797784c12....

Zenith: Acknowledged by client.

4.3 Low Risk

A total of 26 low risk findings were identified.

[L-1] Swap steps with zero movement after rounding can consume exact input without explicit accounting ownership

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target:

- tick.rs
- curve_math.rs
- ix_swap.rs

Description:

Exact-input swaps can consume a positive input amount even when the AMM price does not change and no output is produced.

When a swap step does not have enough input to reach the target tick price, `calculate_filled_market_making_amount_exact_in` computes an intermediate price with `get_next_sqrt_price_from_input` and then unconditionally treats the full input as consumed:

```
let next_sqrt_price = get_next_sqrt_price_from_input(
 current_sqrt_price,
 current_liquidity,
 amount_in,
 swap_for_y,
)?;

(next_sqrt_price, amount_in, 0)
```

However, because `sqrt_price` is stored as an integer Q64.64 value, a small positive input can round bAcknowledged to the original price:

```
next_sqrt_price == current_sqrt_price
```

The output amount is then calculated from a zero price delta and becomes zero:

```
get_delta_amount_y(actual_next_sqrt_price, current_sqrt_price,
 current_liquidity, Rounding::Down)
```

The step result is therefore:

```
amount_in consumed > 0
next_sqrt_price = current_sqrt_price
out_amount = 0
amount_left = 0
```

Because `amount_left` is zero, the outer swap instruction's full-consumption check passes. The user's input is transferred into the reserve, but it is not credited to market-making fee growth, protocol fees, partner fees, limit-order fees, or user output. It becomes reserve surplus without explicit accounting ownership.

This can happen when the user's original input is dust, or when a multi-step swap leaves a dust-sized remainder for the final AMM step.

This differs from Uniswap V3's swap-step accounting. In the exact-input path, when the target price is not reached, Uniswap recomputes the actual input from the resulting price movement and assigns the remaining input to `feeAmount`:

```
if (exactIn && sqrtRatioNextX96 ≠ sqrtRatioTargetX96) {
 feeAmount = uint256(amountRemaining) - amountIn;
}
```

If the rounded next price equals the current price, the recomputed `amountIn` is zero and the whole remaining input is accounted for as a fee. DLMM Pro instead keeps the full amount as consumed swap input while producing no output and no fee accounting entry.

Recommendations:

It is recommended to determine explicit ownership and accounting for this class of dust input.

Meteora: Acknowledged. Added a comment documenting this behavior in PR-170

Zenith: Verified.

[L-2] Relative tick shifts can invalidate preselected tick-array accounts

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target:

- ix_open_limit_order.rs#103
- tick_array_utils.rs#51

Description:

This issue affects the `open_limit_order` instruction when relative ticks are used, in both pools with liquidity-mining rewards and pools without them. Its handler calls `normalize_amount_in_ticks` to resolve relative order offsets from the pool's execution-time active tick, then passes the resulting tick IDs to `init_tick_array_if_needed`.

However, clients must choose the tick-array accounts before execution, while an active-tick movement permitted by `max_slippage` can move a normalized order into a different array and `init_tick_array_if_needed` matches required arrays to supplied accounts only by position.

A relative order can therefore satisfy its slippage and side constraints but still revert with `InvalidTickArray`, allowing ordinary price movement or deliberate boundary movement to make the order fail and charge the submitter a transaction fee.

Example Scenario:

1. With tick spacing `1`, **array 0 contains ticks** *`0`*-*`127`*, and **array 1 contains ticks** *`128`*-*`255`*. The client observes active tick `127` and submits a relative ask offset of `+1` with `max_slippage = 1`.
2. The allowed active ticks and resulting order ticks are:

Active tick at execution	Order tick (active tick + 1)	Required array
126	127	Array `0`
127	128	Array `1`
128	129	Array `1`

The client supplies `[array 0, array 1]` to cover all three possibilities.

3. The active tick remains `127`, so slippage passes and the order resolves to tick `128`, in **array 1**.
4. The program checks the first supplied account, **array 0**, against the required **array 1** and fails with `InvalidTickArray`, despite array `1` also being supplied.

```
id: tick_spacing_aligned_current_id
 .safe_add(lo_tick.id.safe_mul(tick_spacing as i32)?)?,

let tick_array_account_info = remaining_accounts
 .get(index)
 .ok_or(PoolError::InvalidTickArray)?;
```

Recommendations:

Allow clients to supply candidate arrays for every boundary reachable within `max_slippage`, then resolve the execution-time required subset by validated PDA and tick index instead of positional correspondence alone.

```
let tick_array_account_info = find_validated_tick_array(
 remaining_accounts,
 pool_pk,
 tick_index,
)?;
```

Meteora: Resolved with PR-163

Zenith: Verified.

[L-3] Prefunded tick-array PDAs can exhaust the instruction trace

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target:

- modify_account_info.rs#89

Description:

`create_pda_account` creates an already-funded PDA by separately transferring rent, allocating data, and assigning ownership.

However, predictable tick-array PDAs can be prefunded so each lazy initialization takes three System Program CPIs instead of one, followed by another rent transfer for dynamic tick data.

An external party can exhaust Solana's instruction-trace limit and make otherwise valid batched position openings fail.

Example Scenario:

1. A user prepares an `open_position` with 14 ranges spanning 14 previously unused tick arrays.
2. A third party transfers `Rent::minimum_balance(0)` lamports to each predictable tick-array PDA before submission.
3. Each array consumes four nested instructions: transfer, allocate, assign, and the dynamic-data rent transfer.
4. The arrays alone consume 56 trace entries, and the position, NFT, associated-token, metadata, and event instructions push the transaction beyond the 64-entry limit.

```
if new_pda_account.lamports() > 0 {
 invoke(&system_instruction::transfer(/* ... */), /* ... */)?;
 invoke_signed(&system_instruction::allocate(/* ... */), /* ... */)?;
 invoke_signed(&system_instruction::assign(/* ... */), /* ... */)?;
}
```

Recommendations:

Detect uninitialized tick arrays before building the main transaction. If many arrays are missing, initialize them first with `initialize_tick_array` (splitting them across transactions if necessary), then submit the original operation.

Meteora: Acknowledged, it is relied on client side.

Zenith: Acknowledged by client.

[L-4] Frozen reward vaults can permanently block position closure

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target:

- ix_claim_reward.rs#127
- position_reward.rs#43

Description:

`handle_claim_reward` clears each range's pending reward and transfers the aggregated amount from the configured reward vault.

However, it provides no way to clear pending rewards when the reward vault is frozen, so the failed transfer rolls bAcknowledged every pending-balance reset.

Because `PositionWithReward::is_empty` rejects ranges with pending rewards, a permanently frozen vault can leave an otherwise empty position unclosable and strand its NFT and account rent.

Example Scenario:

1. A position accrues `100` reward units and later removes all liquidity and claims all trading fees.
2. The reward mint authority freezes the configured reward vault and then becomes unavailable or renounces the ability to thaw it.
3. The owner calls `claim_reward`; `claim_reward` temporarily clears the `100` pending units, but `transfer_from_pool` fails because the source vault is frozen, rolling bAcknowledged the reset.
4. The owner calls `close_position`, but `PositionWithReward::is_empty` sees the persistent `100`-unit pending balance and rejects closure indefinitely.

```
for position in dynamic_position.dynamic.iter_mut() {
 let reward = position.claim_reward(reward_index);
 total_reward = total_reward.safe_add(reward)?;
```

```
}

if total_reward > 0 {
 transfer_from_pool(
 // frozen reward vault is used as the source
 )?;
}

for reward in self.rewards.iter() {
 if reward.pending_amount ≠ 0 {
 return Ok(false);
 }
}
```

Recommendations:

Add an explicit frozen-vault skip option equivalent to DAMM-v2 that permits the owner or a delegate with unrestricted `ClaimReward` permission to clear pending rewards without transferring them.

```
if reward_vault.is_frozen() && skip_reward {
 assert_unrestricted_claim_reward_authority()?;
} else {
 transfer_from_pool(/* accounts */, total_reward, /* remaining args */)?;
}
```

Meteora: Acknowledged initially and then addressed with 173.

Zenith: This was initially acknowledged, but later mitigated with PR #173, which reduces the exposure by restricting permissionless (pool creator) initialization of freezable reward mints.

- Pool creators cannot initialize rewards with a mint that has freeze authority unless it has a valid token badge or is whitelisted (wrapped SOL / USDC).
- Authorized operators may initialize rewards with freezable mints, subject to the remaining mint-validation checks.

[L-5] Duplicate tick ranges bypass the stale-index guard

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target:

- ix_add_liquidity_imbalance.rs#69

Description:

`AddLiquidityImbalanceParameters::validate` tries to reject stale position indexes by comparing the selected entry's lower and upper ticks with `expected_tick_range`.

However, duplicate tick ranges are permitted and range removal compacts the array, so another accounting entry with identical bounds can move into the stale index and satisfy the check.

A delegated transaction can add liquidity to the wrong logical tranche, affecting its independent lock, vesting, fee, and reward state.

Example Scenario:

Initial array:

- Index 0: Q, empty.
- Index 1: R1, ticks `[-100, 100]`.
- Index 2: R2, ticks `[-100, 100]`.

A delegate signs a deposit targeting index 1 and expecting `[-100, 100]`. Before it lands, the owner removes Q:

- R1 moves to index 0.
- R2 moves to index 1.
- The boundary check still passes.
- The delegate's funds are credited to R2 rather than R1.

```
require!(
 base_position.tick_lower_id == self.expected_tick_range.tick_lower_id
 && base_position.tick_upper_id ==
 self.expected_tick_range.tick_upper_id,
```

```
 PoolError::InvalidParameters
);
```

Recommendations:

Bind the request to an immutable range identifier or to a position-array revision incremented whenever entries are expanded or removed.

```
require!(
 position.range_revision == self.expected_range_revision,
 PoolError::InvalidParameters
);
```

Meteora: Resolved with PR-128.

Zenith: Verified.

[L-6] Bonding target validation accepts prices above the highest spacing-aligned tick

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target:

- activation_bonding_curve.rs#47

Description:

`PodAlignedActivationBondingCurve::validate_initialize` validates that a bonding target remains below the global maximum tick price.

However, it ignores the pool's `tick_spacing`, so it accepts targets above the highest tick at which positions or limit orders can legally place liquidity.

The pool can never reach the accepted target, permanently preventing public liquidity additions and removals and potentially consuming the canonical launch-pool address.

Example Scenario:

1. A launch pool uses `tick_spacing = 256`, whose highest usable tick is `443392` because `MAX_TICK_ID = 443636` is not spacing-aligned.
2. The creator selects an initial square-root price and `target_increase_bps` that produce a target at tick `443433`.
3. Validation accepts the target because tick `443433` remains below the global maximum.
4. Liquidity can extend only to tick `443392`, so swaps cannot move the price high enough to set `is_reach_target`.
5. Public liquidity additions and removals remain disabled permanently.

```
require!(
 target_sqrt_price < get_sqrt_price_from_tick_id(MAX_TICK_ID),
 PoolError::InvalidParameters
);
```

Recommendations:

Pass `tick_spacing` into bonding activation validation and reject targets above the square-root price of the highest spacing-aligned usable tick.

```
let spacing = i32::from(tick_spacing);
let max_usable_tick = MAX_TICK_ID.div_euclid(spacing) * spacing;
require!(
 target_sqrt_price ≤ get_sqrt_price_from_tick_id(max_usable_tick),
 PoolError::InvalidParameters
);
```

Meteora: Resolved with @77d086ab58....

Zenith: Verified.

[L-7] Stale bitmap indexes can remove an unintended empty range

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target:

- ix_remove_position.rs#154

Description:

`remove_position_with_bitmap` removes selected empty position ranges and compacts the surviving entries by current array index.

However, the request does not bind bitmap indexes to the tick ranges observed by the caller, so an earlier removal can shift a different empty range into a selected index before the transaction lands.

A stale authorized transaction can delete an unintended strategy range, change every subsequent range index, and disrupt queued position-management operations.

Example Scenario:

1. A position stores empty ranges A, B, and C at indexes `0`, `1`, and `2`, with B and C covered by the same tick-array account.
2. A caller prepares bitmap `0b010` to remove B.
3. Before it lands, another authorized transaction removes A, compacting B to index `0` and C to index `1`.
4. The stale transaction now removes C, and tick-array validation still succeeds because B and C use the same tick-array account.

```
for i in 0..positions.len() {
 if bitmap & (1 << i) != 0 {
 let position = &mut positions[index];
 require!(position.is_empty()?, PoolError::PositionIsNotEmpty);
 positions[index..].rotate_left(1);
 } else {
 index += 1;
```

```
  }
}
```

Recommendations:

Require the caller to supply an expected position-array revision or expected tick ranges for every selected bitmap bit and validate the binding before mutating the array.

```
require!(
 position.range_revision == parameters.expected_range_revision,
 PoolError::InvalidParameters
);
```

Meteora: Resolved with PR-128.

Zenith: Verified.

[L-8] Amount-derived liquidity ignores executable tick caps

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target:

- curve_math.rs#221

Description:

`get_liquidity_from_amount_y` computes the maximum liquidity affordable by a token-Y amount over a square-root-price range.

However, `add_liquidity_imbalance` forwards that amount-derived result without clamping it to the executable per-tick gross-liquidity headroom or signed-liquidity limit.

A token amount that can fund valid liquidity can therefore derive an excessive value and make the imbalance deposit revert, leaving some valid deposits unavailable through this instruction.

Example Scenario:

1. A position uses adjacent ticks `-443636` and `-443635` at tick spacing `1`, whose square-root-price difference is `214747`.
2. The caller supplies one raw token-Y unit, so the helper derives liquidity `1584573320795813042619336276789748`.
3. The per-tick maximum is only `383514844834609487117690504987493`, and that maximum still costs exactly one raw token-Y unit.
4. `add_liquidity_imbalance` forwards the larger result and reverts with `LiquidityIsTooHigh`, although an ordinary liquidity addition using the legal maximum succeeds.

```
let result = prod.safe_div(delta_sqrt_price)?;
Ok(Some(U320::from(result)))

let liquidity: u128 = try_cast(liquidity)?;
```

Recommendations:

Clamp the amount-derived result to the remaining gross-liquidity headroom at both boundary ticks and to `i128::MAX` before forwarding it to the common liquidity handler.

```
let executable_cap = lower_headroom
 .min(upper_headroom)
 .min(i128::MAX as u128);
let liquidity = amount_limited_liquidity.min(U320::from(executable_cap));
```

Meteora:

Acknowledged: - The case cross gross liquidity is very rare, especially when we downscale liquidity to u128

- Recalculate deposited liquidity based on remaining gross liquidity is confused for integration, I would rather return error instead of make it through with lesser liquidity

Zenith: Acknowledged by client.

PR-104 makes this failure much less likely by increasing the effective per-tick liquidity capacity by `2^64`. It can still occur when a tick has insufficient remaining headroom, but the original scaling made that limit far easier to hit.

[L-9] Zero-net transfer-fee swaps can advance pool state without a fill

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target:

- ix_swap.rs#250

Description:

`handle_swap` converts the caller's gross input into the amount remaining after a Token-2022 transfer fee before executing an exact-input swap.

However, it accepts zero net input, allowing the zero-liquidity path to advance to the next initialized tick while the full-consumption and zero minimum-output checks still pass.

A transfer-fee-only swap can change the pool price, tick, liquidity, and bonding-curve activation state without depositing spendable input into the reserve or producing output.

Example Scenario:

1. A Token-2022 input mint charges a maximum fee of `100` units on a `100`-unit transfer.
2. The pool has zero current liquidity, and the next initialized tick is the bonding-curve activation target.
3. A caller swaps `100` units with minimum output `0`, producing zero net input.
4. The swap jumps to the next tick, commits the new pool state, and reaches the activation target despite producing zero output.

```
let transfer_fee_excluded_input_amount =
 calculate_transfer_fee_excluded_amount(input_mint,
 input_amount)?.amount;

let mut swap_final_result = SwapFinalResult {
 amount_left:  transfer_fee_excluded_input_amount,
 ..Default::default()
};
```

Recommendations:

Reject zero input after excluding the Token-2022 transfer fee before updating the oracle or pool state.

```
require!(
 transfer_fee_excluded_input_amount > 0,
 PoolError::InvalidParameters
);
```

Meteora: Acknowledged, we won't block this, because pool's fee can also take all input, so the fix is not really consistent.

Zenith: Acknowledged by client.

[L-10] Fresh tick arrays undercount the per-instruction reallocation limit

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target:

- tick_array_utils.rs#121

Description:

`init_tick_array_if_needed` creates missing tick-array accounts and expands them to hold every requested boundary tick.

However, its reallocation-limit check subtracts the newly created 64-byte header even though Solana measures total growth from the account's zero-byte size at instruction entry.

A valid fresh-array request can pass the protocol check but fail with `InvalidRealloc`, making the outcome depend on whether the array was initialized previously.

Example Scenario:

1. A non-liquidity-mining pool has no tick array at index `0`.
2. A user opens `29` ranges requiring `58` unique ticks in that array.
3. The final account size is `10,272` bytes, but the helper checks only the `10,208`-byte growth after creating the header.
4. Solana rejects the actual `10,272`-byte instruction growth because it exceeds the `10,240`-byte limit.
5. The same expansion succeeds after the base array is initialized separately.

```
let old_space = TickArray::space(old_length, is_support_liquidity_mining);
let new_space = TickArray::space(new_length, is_support_liquidity_mining);

realloc_expand_account(tick_array_account_info, old_space, new_space)?;
```

Recommendations:

Capture the account's data length before creation and validate final growth against that instruction-entry length.

```
let pre_mutation_space = tick_array_account_info.data_len();
let growth_from_entry = new_space.safe_sub(pre_mutation_space)?;
require!(
 growth_from_entry ⩽ MAX_PERMITTED_DATA_INCREASE,
 PoolError::ReallocExceedMaxLengthPerInstruction
);
```

Meteora: Resolved with @fda76f03e5...

Zenith: Verified.

[L-11] Uninitialized bitmap extensions block inward liquidity searches

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target:

- mod.rs#361

Description:

`get_non_zero_liquidity_next_index` searches the internal and optional extension bitmaps for the next tick array containing liquidity in the swap direction.

However, when the starting index is outside the internal bitmap and the extension is uninitialized, it returns `None` even when the direction points inward toward a populated internal bitmap.

Pools initialized outside the internal range without an extension can reject swaps toward reachable internal liquidity with `LiquidityIsZero`.

Example Scenario:

1. A tick-spacing-`1` pool is initialized at tick-array index `512` without a bitmap extension.
2. A position wholly inside internal array `511` adds liquidity and sets internal bitmap bit `511`.
3. A downward `swap_for_y` starts its search at array `512` and should find array `511`.
4. The function returns `None` because the extension is uninitialized, so the swap raises `LiquidityIsZero`.

```
if self.is_overflow_default_tick_array_bitmap(from_tick_index) {
 if self.is_bitmap_extension_initialized() {
 // extension traversal
 } else {
 Ok(None)
 }
}
```

Recommendations:

When no extension is initialized, continue from internal boundary `511` for positive overflow moving downward and `-512` for negative overflow moving upward.

```
let (min_internal, max_internal) = Self::bitmap_range();
if swap_for_y && from_tick_index > max_internal {
 return self
 .get_non_zero_liquidity_tick_array_index_from_internal_to_extension(
 true,
 max_internal,
 tick_array_bitmap_extension,
 );
}
if !swap_for_y && from_tick_index < min_internal {
 return self
 .get_non_zero_liquidity_tick_array_index_from_internal_to_extension(
 false,
 min_internal,
 tick_array_bitmap_extension,
 );
}
```

Meteora: Resolved with PR-152.

Zenith: Verified.

[L-12] Fully filled limit orders can be blocked by remove-liquidity activation gates

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target:

- ix_cancel_limit_order.rs
- pool_utils.rs
- tick.rs

Description:

Limit-order cancellation is used both to withdraw unfilled maker amounts and to claim the assets received by already-filled orders. The order status is determined later from the tick's `order_age`:

```
if age == self.order_age {
 Ok(LimitOrderStatus::NotFilled)
} else if age.safe_add(1)? == self.order_age {
 Ok(LimitOrderStatus::PartialFilled)
} else if age.safe_add(2)? ⩽ self.order_age {
 Ok(LimitOrderStatus::Fulfilled)
}
```

For a fully filled order, the cancel result has no unfilled maker amount left to withdraw:

```
LimitOrderStatus::Fulfilled ⇒ (lo_tick.amount, 0),
```

However, `handle_cancel_limit_order( ... )` applies the pool's remove-liquidity activation gate before it knows whether the order is not filled, partially filled, or fully filled:

```
validate_pool_can_remove_liquidity(&pool_account_info,
 &ctx.accounts.nft_account.owner)?;

let result = cancel_limit_order::<Tick>(
 remaining_accounts,
 limit_order_ai,
```

```
  pool_pk,
  &ctx.accounts.pool,
  &ctx.accounts.tick_array_bitmap_extension,
  &tick_ids,
 )?;
```

The remove-liquidity gate checks `can_remove_liquidity(...)` for the pool's activation configuration. This is appropriate when cancellation withdraws unfilled maker liquidity, but it also blocks a fully filled order where cancellation only claims the proceeds that were already earned.

In TimeBased activation mode, the pool can be configured so there is a period where users are allowed to add liquidity but are not yet allowed to remove liquidity. During this window, the same remove-liquidity gate also prevents fully filled limit orders from being claimed.

As a result, a maker whose limit order has already been fully filled can be prevented from claiming their output tokens until the pool's remove-liquidity activation condition is satisfied, unless they are the configured whitelisted remove-liquidity address.

Recommendations:

Determine each limit order's status before applying the remove-liquidity gate.

Apply the gate only when cancellation returns unfilled maker amounts. Fully filled orders should be allowed to claim their proceeds even while liquidity removal is still restricted.

Meteora: Acknowledged

[L-13] BondingCurve activation assumes the target token is token_x and reverses the price milestone when it is token_y

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target:

- activation_bonding_curve.rs
- ix_initialize_pool_with_dynamic_config.rs
- ix_initialize_launch_pool.rs

Description:

BondingCurve activation is used by launch pools to keep liquidity operations gated until the launched token reaches a configured price milestone. The activation state stores only `target_increase_bps`; it does not store which mint is the launched token or which price direction should be monitored.

The implementation always interprets the milestone as an increase in the pool's internal `sqrt_price`:

```
let target_sqrt_price: u128 = try_cast(
 U192::from(initial_sqrt_price)
 .safe_mul(U192::from(Into::<u64>::into(self.target_increase_bps)))?
 .safe_div(U192::from(MAX_BASIS_POINT))?,
)?;

if current_sqrt_price ≥ target_sqrt_price {
 self.is_reach_target = 1;
}
```

This condition is correct only when the target launch token is `token_x`, because the pool price is represented as `token_y / token_x`. In that layout, an increase in `sqrt_price` means that `token_x` is worth more units of `token_y`.

However, pool initialization does not enforce that the launch token is `token_x`. The dynamic and launch pool initializers only require the two mints to be different and then store the caller-provided `token_x_mint` and `token_y_mint`:

```
constraint = token_x_mint.key() != token_y_mint.key()

pool.initialize(
  ...,
 ctx.accounts.token_x_mint.key(),
 ctx.accounts.token_y_mint.key(),
  ...
);
```

Configuring the launched or target token as `token_y` can be a valid product requirement rather than an obviously invalid setup. For example, the protocol supports `CollectFeeMode::OnlyY`, so pool creators may intentionally choose a token layout where the relevant fee or launch-token accounting is tied to `token_y`.

If the launched token is instead configured as `token_y`, the economic meaning of the check is reversed. A `sqrt_price` increase means `token_y` is becoming cheaper relative to `token_x`, while a rally in `token_y` would decrease `sqrt_price`. In that configuration, the BondingCurve milestone can be reached when the launched token moves in the wrong direction, or it may fail to activate when the launched token actually reaches the intended price milestone.

This can cause launch pools to open add/remove liquidity permissions at an unintended market state, or keep them closed despite the intended target-token price being reached. The issue is especially relevant for new-token launches because the BondingCurve mechanism is meant to encode the launch token's price milestone, but the on-chain state does not bind that milestone to a specific mint direction.

Recommendations:

Add an explicit price direction or target mint to the activation parameters. Then apply the correct comparison for the selected target token:

- target `token_x` price increase: activate when `sqrt_price` increases to the target;
- target `token_y` price increase: activate when `sqrt_price` decreases to the inverse target.

Meteora: Acknowledged with code comment PR in PR-149.

Zenith: Acknowledged by client.

[L-14] Rounding in LP accounting may reduce rewards and leave unaccountable residual dust in the vault.

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target:

- position_utils.rs
- ix_refresh_fees_and_rewards.rs
- ix_claim_fee.rs
- ix_claim_reward.rs
- position.rs
- position_reward.rs
- pool_reward.rs

Description:

LP fee and reward accounting is checkpoint-frequency dependent because fractional accrual is truncated when a position is refreshed, and the truncated remainder is not stored.

During position updates, fees are calculated from growth deltas using integer division by 2^128:

```
let fee_delta_x =
 U320::from(total_position_liquidity)
 .safe_mul(U320::from(growth_delta_x))? >> 128;

update.fee_growth_checkpoint_x = fee_growth_inside_x;
update.fee_pending_x = position
 .base()
 .fee_pending_x
 .safe_add(try_cast(fee_delta_x)?)?;
```

Reward accrual follows the same pattern:

```
let amount_owed_delta_u320 = U320::from(total_position_liquidity)
 .safe_mul(U320::from(reward_growth_delta))?
 >> 128;
```

```
update.growth_inside_checkpoint = reward_growth_inside;
update.pending_amount = curr_reward_info
 .pending_amount
 .saturating_add(amount_owed_delta);
```

Only the integer part is added to `fee_pending_x`, `fee_pending_y`, or `PositionRewardInfo.pending_amount`. The low 128 bits of the product are discarded. At the same time, the fee or reward growth checkpoint is advanced to the current growth value.

This means the discarded fractional part cannot be recovered by a later claim. `claim_fee` only transfers the already-stored integer pending fees and resets them to zero. `claim_reward` similarly transfers the already-stored integer `pending_amount` and clears it. Neither path has enough information to reconstruct fractional accrual from previous checkpoint intervals.

As a result, splitting one accounting interval into multiple refreshes can reduce the final amount credited to an LP:

```
single refresh: floor(a + b + c)
many refreshes: floor(a) + floor(b) + floor(c)
```

The second value can be smaller because each checkpoint discards its own fractional tail. The loss from a single checkpoint is small in isolation, but it can become visible for small positions, low-emission rewards, frequent refreshes, or many positions. In the extreme case, repeated refreshes can keep crediting no integer reward while still advancing the checkpoint.

For reward tokens with fewer decimals, this effect becomes easier to observe because each minimum transferable unit represents a larger amount of the UI token. The practical impact depends on the reward emission rate, the position's share of pool liquidity, and how often the position is refreshed or modified. Large positions or high reward emissions are less sensitive, while small positions under frequent checkpointing are the main risk case.

For rewards, the corresponding tokens remain in the reward vault, but the program no longer has per-position accounting that can assign those tail amounts to the LPs that produced them. `withdraw_ineligible_rewards` only accounts for periods where pool liquidity was zero; it does not reconstruct rounding remainders lost by position-level checkpoint updates. Therefore reward vaults may retain residual dust that is not claimable by the LPs according to stored position state.

Recommendations:

Preserve and roll forward fractional fee and reward remainders so checkpoint frequency does not change final LP accrual.

Define an explicit finalization or sweep policy for residual reward-vault dust.

Meteora: Acknowledged

[L-15] Zero fill fee splitting can credit future limit order makers

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target:

- fee_sharing_math.rs
- tick.rs
- ix_swap.rs

Description:

`split_fees( ... )` can assign a trading fee to the limit-order fee bucket even when no market-making liquidity or limit-order liquidity was actually filled.

In the exact-input path, the program can collect a non-zero fee before the effective swap amount is applied. Under high-fee and small-input conditions, the amount left after fees can be zero. The fill result then contains no real fill:

```
FilledAmountResult {
 amount_in: 0,
 out_amount: 0,
 mm_amount_in: 0,
 mm_amount_out: 0,
 processed_order_amount_out: 0,
 open_order_amount_out: 0,
 ...
}
```

However, `split_fees( ... )` still tries to split the non-zero `trading_fee` using the filled amount as the share denominator:

```
let (mm_share, total_share) = if fee_setting.fees_on_input {
 (
 filled_amount_result.mm_amount_in,
 filled_amount_result.amount_in,
 )
} else {
 (
```

```
 filled_amount_result.mm_amount_out,
 filled_amount_result.out_amount,
 )
};

let (total_mm_fee, total_limit_order_fee) =
 calculate_shared_amounts(trading_fee, mm_share, total_share,
 Rounding::Up)?;
```

When both `mm_share` and `total_share` are zero, `calculate_shared_amounts( ... )` returns `(0, trading_fee)` because it exits early on `numerator == 0`. As a result, the entire fee becomes `total_limit_order_fee`, even though no limit order was filled.

The tick then applies the fee to the current direction's limit-order fee bucket:

```
self.limit_order_fee_ask_side = self
 .limit_order_fee_ask_side
 .safe_add(swap_result.limit_order_fee)?;
```

or:

```
self.limit_order_fee_bid_side = self
 .limit_order_fee_bid_side
 .safe_add(swap_result.limit_order_fee)?;
```

Because no actual order was filled, there is no maker that earned this fee. If the tick has no filled limit-order accounting on that side, the fee is not immediately claimable by anyone. The fee bucket can nevertheless remain on the tick and later be distributed to a future limit order that fills and cancels on that side. This can cause a later, unrelated limit-order maker to receive fees paid before that maker provided any liquidity.

There is also a loss-of-accounting case. Tick retention is based on `ref_count`; it does not require `limit_order_fee_ask_side` or `limit_order_fee_bid_side` to be zero. If the tick later becomes unreferenced and is compacted out of the tick array, the stray limit-order fee accounting can be removed with the tick state rather than being paid to a defined recipient.

Recommendations:

If the filled amount is zero, do not credit any limit-order fee bucket.

Handle zero-fill fee cases explicitly, either by rejecting the swap step or by routing the fee to a clearly defined non-limit-order recipient.

Meteora: Resolved with PR-146.

Zenith: Verified.

[L-16] Legacy SPL mints bypass freeze-authority validation, allowing a malicious issuer to freeze funded pool reserves or reward vaults

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target:

- token.rs

Description:

The `is_supported_mint(...)` helper is used by pool initialization and reward initialization flows to decide whether a mint can be used permissionlessly or must first receive a protocol-controlled `TokenBadge`.

For Token-2022 mints, the helper rejects mints with a freeze authority. However, legacy SPL Token mints return `Ok(true)` immediately before the freeze-authority check is reached:

```
pub fn is_supported_mint(mint_account: &InterfaceAccount<Mint>) →
 Result<bool> {
 let mint_info = mint_account.to_account_info();
 if *mint_info.owner == Token::id() {
 return Ok(true);
 }

 // ...

 if mint_account.freeze_authority.is_some() {
 return Ok(false);
 }
}
```

As a result, an ordinary user can create a legacy SPL mint with a retained freeze authority and initialize a pool or reward vault without TokenBadge review. The pool reserve or reward vault is then a normal token account for that mint, and the external mint freeze authority can freeze it independently of the protocol's PDA authority, mint-account constraints, checked transfers, and atomic rollbAcknowledged guarantees.

Once third-party liquidity, swap proceeds, fees, or reward funds enter the affected vault,

the malicious mint issuer can freeze the vault. The issuer can also remove the mint's freeze authority after freezing, which can make the frozen vault impossible to thaw. Since the protocol does not appear to provide a vault-replacement or protocol-controlled thaw transition, any operation that needs the frozen reserve or reward vault can remain unavailable, including swaps, liquidity removal, fee claims, and reward claims.

Recommendations:

Apply the freeze-authority validation to legacy SPL Token mints as well as Token-2022 mints. A mint with `freeze_authority.is_some()` should not be treated as permissionlessly supported.

Meteora: Acknowledged with code comment in @3c87dcae44....

Zenith: This was later mitigated for reward mints in PR-173,

- To preventing creators from initializing reward mint with freeze authority, unless the reward mint has token badge or is whitelisted (SOL/USDC).
- For authorized operators they are allowed to initialize any reward mint regardless of freeze authority

Note that pool initialization still permit freezable mints, so the pool-reserve exposure remains acknowledged.

[L-17] Pools created through `initialize_launch_pool` can omit activation gating

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target:

- ix_initialize_launch_pool.rs#168

Description:

`initialize_launch_pool` creates a launch-style pool whose public operations are expected to be controlled by activation rules.

However, it reuses the generic dynamic-parameter validator, which accepts no activation extension and also accepts two `Open` activation modes.

The resulting `LaunchPool` can permit swaps and public liquidity operations immediately despite being represented as launch-gated.

Example Scenario:

1. An initializer submits `extensions` as `0`, with no activation point and no activation state.
2. Generic validation accepts the parameters and the handler stores `PoolType::LaunchPool` without a `pool_activation` value.
3. A public user immediately swaps and adds liquidity because downstream checks apply activation restrictions only when activation state exists.

```
parameters.validate()?;

if is_support_extension(extensions, PoolExtension::ActivationAccess) {
 // Activation state is initialized only conditionally.
}
```

Recommendations:

Add launch-specific validation that requires the activation extension and state and rejects the `Open` plus `Open` combination for launch pools.

```
require!(
 is_support_extension(self.extensions, PoolExtension::ActivationAccess),
 PoolError::InvalidParameters
);
require!(self.pool_activation.is_some(), PoolError::InvalidParameters);
```

Meteora: Acknowledged with code comment PR @240fe0d166....

Zenith: Acknowledged by client.

[L-18] The permanent delegate can create liquidity through a reserve self-transfer in `add_liquidity`

SEVERITY: Low	IMPACT: Low
STATUS: Open	LIKELIHOOD: Low

Target:

- ix_add_liquidity.rs#58

Description:

`add_liquidity` credits a position and then transfers the required user tokens into the pool reserves.

However, it does not prohibit a user token account from aliasing the corresponding reserve, so a badged Token-2022 mint with `PermanentDelegate` can authorize a successful reserve self-transfer that changes no balance.

The position receives unbAcknowledgeded liquidity that can later withdraw tokens supplied by other LPs and distort active-range swap accounting.

Example Scenario:

1. An operator badges a Token-2022 mint with a permanent delegate and a pool holds 1,000 X tokens in its reserve.
2. The permanent delegate opens an X-only position and calls `add_liquidity` with `user_token_x` equal to `reserve_x` for 100 X tokens of liquidity.
3. The program credits the position before invoking `transfer_checked` from the reserve to itself.
4. Token-2022 accepts the permanent-delegate signature and the self-transfer succeeds without changing the 1,000-token reserve balance.
5. The attAcknowledgeder removes the unbAcknowledgeded liquidity against tokens deposited by other LPs.

```
pub user_token_x: Box<InterfaceAccount<'info, TokenAccount>>,

pub reserve_x: Box<InterfaceAccount<'info, TokenAccount>>,
```

```
transfer_from_user(
 &ctx.accounts.signer.to_account_info(),
 &ctx.accounts.token_x_mint,
 &ctx.accounts.user_token_x.to_account_info(),
 &ctx.accounts.reserve_x.to_account_info(),
 &ctx.accounts.token_x_program.to_account_info(),
 total_amount_x,
 parsed_transfer_hook_accounts.transfer_hook_x,
)?;
```

Recommendations:

Reject all user/reserve aliases before state changes and verify after each CPI that the reserve increased by the expected net token amount.

```
require_keys_neq!(
 ctx.accounts.user_token_x.key(),
 ctx.accounts.reserve_x.key(),
 PoolError::InvalidTokenAccount
);
```

Meteora: Acknowledged, if token badge is issued, then mint is trusted

Zenith: Acknowledged by client.

[L-19] Bitmap indexes in `add_liquidity` and `remove_liquidity` are not bound to expected tick ranges

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target:

- ix_add_liquidity.rs#7
- ix_remove_liquidity.rs#148

Description:

`add_liquidity` and `remove_liquidity` use a bitmap to select position ranges and apply the supplied liquidity values by the array indexes stored in that bitmap.

However, neither request binds a selected index to its expected tick range, even though removing an earlier empty range compacts the array and changes subsequent indexes.

A pending transaction can therefore add liquidity to, or remove liquidity from, a different range than the caller selected. Addition can change the caller's token exposure and fee or reward behavior while still satisfying aggregate token limits. Removal can withdraw from a different funded range if that range has sufficient liquidity and the aggregate slippage checks pass.

Example Scenario — Add Liquidity

1. A position contains empty range A at index 0 and range B at index 1 in the same tick array.
2. A caller prepares an `add_liquidity` transaction with bit 0 set and 100 liquidity units quoted for range A.
3. Before it lands, another transaction removes A and rotates B into index 0.
4. The pending transaction applies the 100 units to B because it validates only the bitmap index and aggregate token limits.

Example Scenario — Remove Liquidity

1. A position stores empty range A at index 0, funded range B at index 1, and funded

range C at index 2.

2. The owner signs a `remove_liquidity` transaction selecting bitmap bit 1 for 100 liquidity units from B.
3. Before that transaction lands, a delegate with range-update permission removes empty range A, compacting the array to B at index 0 and C at index 1.
4. The original transaction applies bit 1 to C and succeeds if C has at least 100 liquidity units and aggregate slippage passes.

```
pub struct AddLiquidityParameters {
 pub max_amount_x: u64,
 pub max_amount_y: u64,
 pub bitmap: u64,
 pub liquidities: Vec<u128>,
}

pub struct RemoveLiquidityParameters {
 pub min_amount_x: u64,
 pub min_amount_y: u64,
 pub bitmap: u64,
 pub liquidities: Vec<u128>,
}

dynamic_position.dynamic[i].apply_position_update(&position_update);
```

Recommendations:

Carry an expected tick range with every selected index for both add and remove operations. Validate all stored lower and upper ticks before calculating or applying any state changes.

```
require!(
 position.base().tick_lower_id == entry.expected_tick_range.tick_lower_id
 && position.base().tick_upper_id ==
 entry.expected_tick_range.tick_upper_id,
 PoolError::InvalidParameters
);
```

Meteora: Resolved with PR-128.

Zenith: Verified.

[L-20] Valid expanded positions can exceed account limits in `refresh_fees_and_rewards`

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target:

- position_utils.rs#157

Description:

`process_refresh_fees_and_rewards` refreshes every non-empty range in a position and loads both boundary ticks for each range.

However, positions can contain up to `45` ranges while refresh provides no bitmap or subset parameter, and each range can require two distinct tick-array accounts.

A protocol-valid expanded position can therefore require more accounts than one Solana transaction permits, making its fresh fees and rewards impossible to checkpoint without changing the position.

Example Scenario:

1. At tick spacing `1`, a user incrementally constructs `30` valid ranges whose lower and upper boundaries occupy `60` distinct tick arrays.
2. The user adds nonzero liquidity to every range in separate bitmap-based transactions.
3. Refresh iterates over all `30` ranges and requires all `60` tick arrays in one call.
4. The tick arrays plus pool, position, NFT account, signer, program, and event-CPI accounts exceed the transaction account limit, so every full refresh fails.

```
let position_length = dynamic_position.length();
for i in 0..position_length {
 // Every non-empty range is refreshed in this call.
}
```

Recommendations:

Add a bitmap that refreshes selected ranges in chunks and validate construction against unique boundary tick-array requirements.

```
pub struct RefreshFeesAndRewardsParameters {
 pub bitmap: u64,
}
```

Meteora: Resolved with PR-141.

Zenith: Verified.

[L-21] A failure on one token side blocks claiming the other side through `claim_fee`

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target:

- ix_claim_fee.rs#68

Description:

`claim_fee` aggregates pending X and Y fees and attempts both transfers in one atomic instruction.

However, callers cannot select one token side, and owner-only delegates must validate both owner associated token accounts before the instruction knows whether both sides have a nonzero balance.

A frozen account, rejecting transfer hook, missing unused associated token account, or other failure on one token can indefinitely prevent collection of valid fees in the other token.

Example Scenario:

1. A position has 100 pending X units and 1,000 pending Y units.
2. The X destination is frozen while the Y destination remains valid.
3. The X transfer fails before the program can complete the Y transfer.
4. Transaction rollbAcknowledged preserves both balances, but the owner cannot claim the healthy 1,000 Y units independently.

```
if pending_fee_x > 0 {
 transfer_from_pool(/* X accounts and amount */)?;
}
if pending_fee_y > 0 {
 transfer_from_pool(/* Y accounts and amount */)?;
}
```

Recommendations:

Make claims directional so each instruction validates, clears, and transfers only the selected token's pending fees and accounts.

```
match token {
 ClaimFeeToken::X ⇒ claim_pending_x()?,
 ClaimFeeToken::Y ⇒ claim_pending_y()?,
}
```

Meteora: Acknowledged, we dont really see that case in reality.

Zenith: Acknowledged by client.

[L-22] Vesting refresh in `lock_permanent_position` can permanently lock more liquidity than intended

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target:

- ix_lock_permanent.rs#50

Description:

`lock_permanent_position` converts a percentage of each range's unlocked liquidity into irreversible permanent liquidity.

However, it refreshes vesting immediately before calculating the amount and accepts no expected balance, maximum amount, or current-point bound.

A transaction landing after a vesting boundary can permanently lock substantially more liquidity than the signer authorized from the state used to construct the transaction.

Example Scenario:

1. A range has `100` unlocked units and `900` vesting units scheduled to unlock at slot `1,000`.
2. At slot `999`, the owner signs a `10,000` basis-point request expecting to permanently lock `100` units.
3. The transaction lands at slot `1,000`, and `process_refresh_vesting` first moves all `900` vested units into the unlocked balance.
4. The instruction permanently locks the resulting `1,000` units with no signed maximum that can reject the changed amount.

```
process_refresh_vesting::<Pool, Position>(&ctx.accounts.pool,
 &ctx.accounts.position)?;
let position_ai = ctx.accounts.position.to_account_info();
lock_permanent_liquidity::<Position>(&position_ai, bps)?
```

Recommendations:

Accept exact per-range amounts with expected unlocked balances and a maximum current point, or add signed maximum-lock bounds that are checked after vesting refresh.

```
require!(current_point ⩽ max_current_point, PoolError::ExceededSlippage);
require!(liquidity ⩽ max_locked_liquidity, PoolError::ExceededSlippage);
```

Meteora: Acknowledged in @24a43d8484....

Zenith: Acknowledged by client.

[L-23] Orders opened through `open_limit_order` lAcknowledged a maximum transfer-fee-inclusive debit

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target:

- ix_open_limit_order.rs#348

Description:

`open_limit_order` grosses up the requested net order amount to cover a Token-2022 transfer fee before depositing it into the reserve.

However, the signed parameters contain no maximum gross debit or transfer-fee bound, so the instruction accepts whatever fee configuration applies at execution.

A mutable fee authority can increase the fee after signing and cause the maker to pay substantially more than intended for the same net order amount.

Example Scenario:

1. A maker signs an order depositing `100` net tokens while the transfer fee is `0%`.
2. Before execution, the fee authority schedules a `90%` fee with a sufficiently large maximum fee.
3. The instruction calculates a gross amount of approximately `1,000` tokens.
4. The user's account is debited `1,000` tokens, `900` are withheld as fees, and only the intended `100` reaches the reserve.

```
let transfer_fee_included_amount =
 calculate_transfer_fee_included_amount(&ctx.accounts.token_mint,
 total_amount)?.amount;

transfer_from_user(/* accounts */, transfer_fee_included_amount,
 /* hook */)?;
```

Recommendations:

Add a maker-specified maximum gross deposit and enforce it after calculating the fee-inclusive amount, with an optional maximum fee-basis-point bound.

```
require!(
 transfer_fee_included_amount <= parameters.max_deposit_amount,
 PoolError::ExceededSlippage
);
```

Meteora: Resolved with PR-127.

Zenith: Verified.

[L-24] SPL multisig owners cannot close limit orders through `close_limit_order`

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target:

- ix_close_limit_order.rs#33

Description:

`close_limit_order` requires the limit-order NFT authority to sign before burning the NFT and closing its token account.

However, the authority is typed as `Signer`, which cannot represent a standard SPL Token multisig whose member accounts sign while the multisig account itself is not a transaction signer.

A freely transferred NFT held by an SPL multisig cannot close its empty order, permanently stranding the limit-order account, NFT mint, and tick references.

Example Scenario:

1. A maker transfers a limit-order NFT into a token account owned by a `2-of-3` SPL Token multisig.
2. The maker cancels the order, leaving all `1,000` order units withdrawn and the order empty.
3. Two multisig members sign a call to `close_limit_order`, but the multisig authority account itself is not marked as a signer.
4. Anchor rejects `owner: Signer` before Token-2022 can validate the two member signatures, so the empty order cannot be closed.

```
token::authority = owner
pub nft_account: Box<InterfaceAccount<'info, TokenAccount>>,

pub owner: Signer<'info>,
```

Recommendations:

Accept an unchecked authority constrained to `nft_account.owner`, distinguish single-key from SPL multisig ownership, and pass a dedicated multisig-signer account slice to the burn and close CPIs.

Meteora: Acknowledged. We don't see anyone use SPL token multisig. It's even rarer than transfer hook.

Zenith: Acknowledged by client.

[L-25] Stale range indices in `expand_position` can redirect a subsequent liquidity deposit

SEVERITY: Low	IMPACT: Low
STATUS: Resolved	LIKELIHOOD: Low

Target:

- ix_expand_position.rs#58

Description:

`handle_expand_position` appends new tick ranges after the position's current range count.

However, it derives the insertion index at execution time without requiring the caller's expected prior range count, while ordinary liquidity instructions select ranges only by bitmap index.

A range-management delegate can insert a range before an owner's transaction lands and redirect the owner's subsequent deposit into the delegate-selected range without holding `AddLiquidity` permission or transfer authority.

Example Scenario:

1. A position has one range, and its owner constructs a transaction that appends range B and deposits `100` liquidity units into bitmap bit `1`.
2. A delegate with only `UpdatePositionRange` permission first appends range A at index `1`.
3. The owner's expansion still succeeds but appends range B at index `2`.
4. The owner's deposit still applies bit `1`, so the `100` liquidity units enter range A if its aggregate token amounts remain below the owner's maxima.

```
let old_range = dynamic_position.length();
let new_range = old_range.safe_add(parameters.tick_ranges.len())?;
expand_position_with_tick_ranges(position_ai, parameters, old_range)?;
```

Recommendations:

Add and validate a monotonically increasing range-layout version in every range-changing and bitmap/index-based instruction, or at minimum require `expand_position` to match an `expected_old_range` and bind each liquidity index to its expected tick range.

Meteora: Resolved with PR-128.

Zenith: Verified.

[L-26] Aggregated limit-order fills can leave per-order rounding dust unclaimable in pool reserves

SEVERITY: Low	IMPACT: Low
STATUS: Acknowledged	LIKELIHOOD: Low

Target:

- tick.rs

Description:

Limit-order liquidity is filled at the tick level, but later settled per order record. During an exact-in swap, the tick calculates the counter-asset input for the aggregate filled amount and rounds up:

```
let total_amount_in =
 Tick::get_amount_in(self.sqrt_price, total_out_amount, swap_for_y,
 Rounding::Up)?;
```

When an owner cancels a filled or partially filled limit order, the program recalculates that owner's proceeds from the individual `fulfilled_amount` and rounds down:

```
let swapped_amount = Tick::get_amount_in(
 self.sqrt_price,
 fulfilled_amount,
 !is_ask_side,
 Rounding::Down,
)?;
```

The protocol therefore collects approximately `ceil(price * total_filled)` from the swapper, but pays `sum(floor(price * order_filled))` to individual order owners. The difference is not trAcknowledgeded as a per-tick or per-order remainder. After cancellation, `lo_tick.cancel_order()` clears the order record, so the rounded dust has no later claim path and remains in the pool reserves as unassigned surplus.

For high-decimal assets, the value is usually small because each order can lose less than one raw unit of the counter asset. The accounting drift becomes more visible for low-decimal assets or highly fragmented dust orders.

This does not make the pool insolvent and does not let the swapper directly withdraw the dust. The issue is that limit-order owners can receive less than the aggregate counter-asset amount collected for their filled orders, while the difference remains in reserves without an explicit accounting bucket.

Recommendations:

It is recommended to define an explicit ownership policy for funds arising from rounding dust.

Meteora: Acknowledged, added PR to make documentation clearer @bcf20f17d3....

Zenith: Acknowledged by client.

4.4 Informational

A total of 18 informational findings were identified.

[I-1] `get_delta_amount_x_unchecked` keeps a narrower intermediate than `get_next_sqrt_price_from_amount_in_x_rounding_up`

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target:

- curve_math.rs#L24
- curve_math.rs#L145

Description:

PR #104 widened the intermediate calculation in `get_next_sqrt_price_from_amount_in_x_rounding_up` from `U256` to `U320` after liquidity was treated as an unscaled `u128` and then shifted left by 64 bits.

```
let sqrt_price = U320::from(sqrt_price);
let liquidity: U320 = U320::from(liquidity) << 64;

let product = U320::from(amount).safe_mul(sqrt_price)?;
let denominator = liquidity.safe_add(U320::from(product))?;
let result = liquidity.safe_mul(sqrt_price)?.div_ceil(denominator);
```

However, the related `get_delta_amount_x_unchecked` helper still keeps the same fixed-point numerator in a `U256` intermediate:

```
let numerator_1: U256 = U256::from(liquidity) << 64;
let numerator_2 = U256::from(upper_sqrt_price.safe_sub(lower_sqrt_price)?);
let prod = numerator_1.safe_mul(numerator_2)?;
```

At the type level, this product can require up to 320 bits:

```
(liquidity << 64) * (upper_sqrt_price - lower_sqrt_price)

u128 shifted by 64 bits => up to 192 bits
sqrt-price delta => up to 128 bits
product => up to 320 bits
```

Under the current production constraints, this wider numerator is not reachable through valid protocol flows and is not expected to have any accounting impact: the supported liquidity caps, reachable tick ranges, and `u64` token-amount boundaries prevent valid operations from requiring an intermediate wider than `U256`. The concern is only helper-level consistency and maintainability: at the type level, `(liquidity << 64) * (upper_sqrt_price - lower_sqrt_price)` can be wider than 256 bits.

Recommendations:

Use `U320` for the numerator path in `get_delta_amount_x_unchecked`, matching the widened `get_next_sqrt_price_from_amount_in_x_rounding_up`.

Meteora: Resolved with PR-175

Zenith: Verified.

[I-2] Rent reductions can under-refund historical tick-array rent contributions

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target:

- modify_account_info.rs
- tick_array_utils.rs#L99-L106
- tick_array_utils.rs#L248-L264
- ix_close_position.rs
- ix_close_limit_order.rs
- ix_remove_position.rs#L121-L135

Description:

Tick-array expansion and shrink accounting treats rent as a formula based on the current `Rent` sysvar, rather than as a historical amount contributed by the account that created the dynamic tick storage.

When a position or limit order initializes tick records, the program credits the tick-array PDA for the current rent-exempt cost of the newly referenced dynamic bytes:

```
let rent = Rent::get()?;
let overhead_lamports = rent.minimum_balance(0);
let credited_rent = rent.minimum_balance(credit_space);
let credited_rent_without_overhead = credited_rent.safe_sub(
 overhead_lamports)?;

transfer(/* rent_payer → tick_array */, credited_rent_without_overhead)?;
```

When those tick references are later removed, the program recomputes the same formula using the then-current `Rent` sysvar and transfers that amount to the closer:

```
let rent = Rent::get()?;
let overhead_lamports = rent.minimum_balance(0);
let credited_rent = rent.minimum_balance(credit_space);
let credited_rent_without_overhead = credited_rent.safe_sub(
 overhead_lamports)?;
```

```
**pda_account.try_borrow_mut_lamports()? =
 pda_lamports.safe_sub(credited_rent_without_overhead)?;
**rent_receiver.try_borrow_mut_lamports()? =
 rent_receiver.lamports().safe_add(credited_rent_without_overhead)?;
```

The release path is reached by `close_position`, `close_limit_order`, and advanced-position range removal through `collect_rent_from_tick_array`:

```
let credited_space = TickArray::dynamic_data_space(ids.len(),
 is_support_liquidity_mining);
collect_rent_from_realloc_shrink(tick_array_account_info, rent_receiver,
 credited_space)?;
```

No state records how many lamports were actually paid when the dynamic tick records were created. Therefore, if network rent decreases between opening and closing, the closer receives less than the original contribution. The difference remains in the tick-array PDA as surplus. Tick arrays are shrunk bAcknowledged to their base layout but are not closed, so that surplus is not returned to the original contributor through the affected lifecycle.

SIMD-0437 proposes reducing `lamports_per_byte` from `6960` to `696` over five feature-gated steps: `6333`, `5080`, `2575`, `1322`, and `696`. The proposal also states that each feature activation updates the bank and `Rent` sysvar and that existing accounts may reduce their balance to the new minimum. This makes historical rent contributions materially different from later current-rent refunds.

Recommendations:

It is recommended to record the actual amount paid and refund rent based on that recorded contribution.

Meteora: Acknowledged, we will check later if the issue really causes trouble.

Zenith: Verified.

[I-3] Primitive `safe_shl` silently discards overflowing high bits

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target:

- safe_math.rs#80

Description:

The primitive `safe_shl` implementations delegate to Rust's `checked_shl`, which checks only whether the shift count is below the type width. Nonzero high bits can be discarded while the operation still returns `Some`, which `safe_shl` maps to `Ok`.

This is documented Rust shift behavior. The `SafeMath` trait does not document a stronger guarantee that left shifts detect value overflow. All three current primitive call sites widen a `u64` to `u128` before shifting by 64 bits, so their results always fit.

This is an informational API-documentation concern: future callers could mistake `safe_shl` for checked multiplication by a power of two. No current accounting impact is demonstrated from the primitive call sites.

Example Scenario:

1. A calculation holds `1u128 << 127` and calls `safe_shl(1)` to multiply it by two.
2. The mathematical result is `2^128`, which does not fit in `u128`.
3. Primitive `checked_shl(1)` returns `Some(0)`, so `safe_shl` returns `Ok(0)` even though the mathematical result does not fit.

```
fn safe_shl(self, v: $offset) -> Result<$t, PoolError> {
 match self.checked_shl(v) {
 Some(result) => Ok(result),
 None => Err(PoolError::MathOverflow),
 }
}
```

Recommendations:

If silent overflow (truncation of nonzero high bits) is intended, document that primitive `safe_shl` follows Rust's `checked_shl` semantics: it returns `MathOverflow` only when the shift count is greater than or equal to the integer type's bit width, and it may otherwise discard high bits without returning an error. Callers that require lossless arithmetic must ensure the shifted value fits.

If `safe_shl` is instead intended to detect arithmetic overflow, use separate signed and unsigned implementations that check the representable range before shifting, including both lower and upper bounds for signed integers.

Meteora: Resolved with PR-169.

Zenith: Resolved by removing `safe_shl` and the trait's shift-offset parameter from safe_math.rs, replacing all eight calls with explicit shifts. Will manually ensure correct bit types when shifting.

[I-4] Pinned `ruint` version makes wide-integer `safe_shl` miss overflow

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target:

- safe_math.rs#102
- Cargo.lock#1618

Description:

The wide-integer `safe_shl` implementations delegate overflow detection to `ruint::Uint::checked_shl`.

However, the lockfile pins `ruint` 1.17.2, whose pre-1.20.0 shift implementation can return a false-negative overflow flag when complete limbs are discarded.

A future wide-integer call that discards a high limb can therefore return a wrapped value instead of `MathOverflow`, undermining accounting checks that rely on `safe_shl`.

This overflow is not reachable through the current call sites. The price, liquidity, and fee calculations shift constants or `u64` values into sufficiently wide integer types. Reward schedule validation requires the rounded total reward amount to fit in `u64`, and accrual is capped to the configured schedule, keeping the reward-growth shift within `U192`. This is therefore an informational dependency issue with no demonstrated current accounting impact.

Example Scenario:

1. A calculation sets a `U192` value to `1 << 128`.
2. It calls `safe_shl(64)`, whose mathematical result is `1 << 192` and does not fit in `U192`.
3. The affected `ruint` implementation misses the discarded high limb and `checked_shl` returns `Some(0)`.
4. `safe_shl` consequently returns `Ok(0)` instead of `MathOverflow`.

```
checked_impl!(U192, usize);
checked_impl!(U256, usize);
```

```
checked_impl!(U320, usize);
checked_impl!(U512, usize);
```

Recommendations:

Update the lockfile to `ruint` 1.20.0 or later and add a regression test for a whole discarded high limb.

```
let x = U192::ONE << 128;
assert!(x.safe_shl(64).is_err());
```

Meteora: Resolved with PR-169.

Zenith: Resolved by removing `safe_shl` and the trait's shift-offset parameter from safe_math.rs, replacing all eight calls with explicit shifts. Will manually ensure correct bit types when shifting.

[I-5] Rent changes break tick-array credit accounting

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target:

- modify_account_info.rs#7

Description:

`credit_account_for_realloc_expand` funds each tick-array reference using the current rent rate, while `collect_rent_from_realloc_shrink` later derives its refund independently from the then-current rate.

However, the protocol neither funds the full post-reallocation rent shortfall nor records the lamports originally credited for each reference.

A rent-rate increase can make tick-array expansion fail, while a rate change between opening and closing causes users to receive a different refund from their original contribution and can make refunds dependent on historical surplus.

Example Scenario:

1. A 64-byte tick array is funded at `6,333` lamports per effective byte with `1,215,936` lamports.
2. The rent rate returns to `6,960`, and a position adds two 176-byte ticks.
3. The helper transfers `2,449,920` lamports for the new bytes, leaving the account with `3,665,856` lamports.
4. The expanded 416-byte account requires `3,786,240` lamports, so it remains short by `120,384` lamports and the upward reallocation fails.

```
let overhead_lamports = rent.minimum_balance(0);
let credited_rent = rent.minimum_balance(credit_space);
let credited_rent_without_overhead =
 credited_rent.safe_sub(overhead_lamports)?;
```

Recommendations:

Fund the complete post-reallocation minimum-balance shortfall rather than charging only for the newly added bytes.

```
let required = Rent::get()?.minimum_balance(new_space);
let shortfall = required.saturating_sub(account.get_lamports());
```

Meteora: Acknowledged, I think rent will reduce over time (https://solana.com/upgrades/reduced-rent), otherwise it will break all dapp, not only Meteora

Zenith: Acknowledged by client. Rent will reduce over time via SIMD-0437 but they have a safeguard gate (SIMD-0438) to restore the legacy value if state growth becomes a problem. Though likely there would be time to patch and prevent this issue.

[I-6] Ceiling-rounded refunds can underfund carried LP rewards

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target:

- ix_withdraw_ineligible_rewards.rs#90
- ix_set_reward_emission.rs#142
- pool_reward.rs#130

Description:

`calculate_reward_amount` converts fractional Q64 emissions into whole token amounts for campaign funding and empty-liquidity refunds.

However, `withdraw_ineligible_rewards` applies the funding helper's ceiling rounding to outgoing refunds while `set_reward_emission` preserves `reward_growth_global` across campaigns.

Repeated ceiling-rounded refunds can remove tokens bAcknowledgeding fractional LP rewards that later accumulate into a whole claim, leaving the reward vault underfunded.

Practical likelihood is low: the described failure requires fractional rewards to accumulate across at least two campaigns without the position being checkpointed between them. The claim fails only if ceiling-rounded refunds leave the vault unable to cover those accumulated rewards.

Example Scenario:

1. Two one-day campaigns each emit `0.5` base units per second and receive `43,200` base units of funding. A position provides the sole active liquidity for `86,399` of each day's `86,400` seconds; there is no active liquidity for just one second per campaign.
2. Each campaign therefore owes the position `43,199.5` base units, while the exact empty-liquidity refund is only `0.5` base units.
3. After each campaign, `withdraw_ineligible_rewards` rounds that refund up to `1` base unit, retaining only `43,199` base units of that campaign's funding in the vault.
4. The position does not checkpoint or claim between campaigns, so preserved global

growth carries its fractional rewards forward. After both campaigns, its accrued rewards total `86,399` claimable base units.

5. The vault received `86,400` base units and refunded `2`, leaving only `86,398`. The position's claim is therefore short by `1` base unit and fails absent unrelated surplus.

This example has active liquidity for more than `99.99%` of each campaign. The problem depends on fractional refund rounding and carried reward growth, not on liquidity being absent for most of the campaign. Amounts are token base units, not whole tokens.

```
let reward_amount = calculate_reward_amount(
 reward_info.cumulative_seconds_with_empty_liquidity_reward,
 reward_info.emission_per_second,
)?;

pub fn calculate_reward_amount(time_delta: u64,
 emission_per_second: u128) → Result<u64> {
 try_cast(
 U192::from(time_delta)
 .safe_mul(U192::from(emission_per_second))?
 .div_ceil(U192::from(Q64)),
 )
}
```

Recommendations:

Round incoming campaign funding up but outgoing ineligible refunds down, and clear `cumulative_seconds_with_empty_liquidity_reward` even when the floor-rounded refund is zero.

```
let reward_amount: u64 = try_cast(
 U192::from(empty_seconds)
 .safe_mul(U192::from(emission_per_second))?
 / U192::from(Q64),
)?;
reward_info.cumulative_seconds_with_empty_liquidity_reward = 0;
```

Meteora: Resolved with @73be22bb9d....

Zenith: Verified.

[I-7] `mul_shift_96` places the middle-product carry at the wrong bit position

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target:

- tick_math.rs#L317-L354

Description:

`mul_shift_96` is intended to return `(n0 * n1) >> 96` without constructing a 256-bit intermediate. It decomposes each operand into 64-bit limbs:

```
n0 = a * 2^64 + b
n1 = c * 2^64 + d
```

The full product is therefore:

```
n0 * n1 = ac * 2^128 + (ad + bc + high64(bd)) * 2^64 + low64(bd)
```

The implementation computes the middle sum modulo `2^128` and records its overflow in `mid_carry`:

```
let bd_hi = bd >> 64;
let (ad_bc, c1) = ad.overflowing_add(bc);
let (mid, c2) = ad_bc.overflowing_add(bd_hi);
let mid_carry = (c1 as u128) + (c2 as u128);
```

However, a carry out of this middle sum represents `mid_carry * 2^128` within the middle coefficient. Because that coefficient is subsequently multiplied by `2^64`, the carry contributes `mid_carry * 2^192` to the full product and `mid_carry * 2^96` after the final right shift.

The current implementation instead adds `mid_carry` directly to `ac` and then shifts it by only 32 bits:

```
let high = ac.wrapping_add(mid_carry).wrapping_shl(32);
let low = mid >> 32;

high.wrapping_add(low)
```

Consequently, whenever `mid_carry == 1`, the function places the carry at bit 32 instead of bit 96 and returns a value that is too small.

For example, consider the following inputs:

```
let n0 = u128::MAX; // 2^128 - 1
let n1 = 3 * (1u128 << 64) - 1;
```

The correct mathematical result is:

```
(n0 * n1) >> 96 = 237684487542793012776336883711
```

However, the current `mul_shift_96` implementation returns:

```
158456325028528675187087900671
```

The result is smaller than the correct value by `2^96 - 2^32` because the middle-product carry is shifted to the wrong position. The example still satisfies the function's documented precondition: the product is 194 bits, which is below `2^224`, and the shifted result fits in `u128`.

This input cannot occur through the current `get_sqrt_price_positive_tick` call path. For every supported positive tick from `0` through `MAX_TICK_ID` (`443,636`), the reachable ratios and precomputed constants do not cause the middle sum to carry. Exhaustive testing of that range found no difference from an exact wide-integer calculation. Therefore, current production tick-price calculations are unaffected, but the helper would become unsafe if its input range were expanded.

Recommendations:

Place `mid_carry` at bit 96 of the shifted result instead of adding it to `ac` before the 32-bit shift:

```
let high = ac.wrapping_shl(32);
let carry = mid_carry.wrapping_shl(96);
high.wrapping_add(carry).wrapping_add(mid >> 32)
```

Meteora: Resolved with @2ea200b9ee...

Zenith: Resolved. Removes unused carry handling, documents the helper's no-carry precondition, and adds exhaustive tests confirming correctness across all supported positive ticks.

[I-8] Dynamic-fee u24 bounds accept 2^24

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target:

- ix_create_static_config.rs#103

Description:

`DynamicFeeParameters::validate` constrains two dynamic-fee fields to the documented unsigned 24-bit range.

However, both checks accept `1 << 24`, while the maximum unsigned 24-bit value is `(1 << 24) - 1`.

The accepted out-of-range value can be incompatible with clients or serializers that encode these fields in exactly 24 bits.

Example Scenario:

1. A client submits `max_volatility_accumulator` as 16,777,216.
2. The program accepts it because it permits values less than or equal to `1 << 24`.
3. A 24-bit client truncates the value to 0 when encoding or decoding the config.
4. The client and on-chain program derive different dynamic-fee behavior.

```
require!(self.max_volatility_accumulator ⩽ 1 << 24,
 PoolError::InvalidParameters);
require!(self.variable_fee_control ⩽ 1 << 24,
 PoolError::InvalidParameters);
```

Recommendations:

Define an explicit `U24_MAX` constant as `(1u32 << 24) - 1` and use it for both bounds.

```
const U24_MAX: u32 = (1u32 << 24) - 1;
```

Meteora: Resolved with @1bd9ca9b43....

Zenith: Verified.

[I-9] Expanded arrays fail idempotent initialization

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target:

- ix_initialize_tick_array.rs#17

Description:

`handle_initialize_tick_array` creates a tick-array PDA or treats an existing array as an idempotent initialization.

However, its `init_if_needed` constraint requires every existing account to retain the initial 64-byte size even though valid tick arrays expand as ticks are added.

Reinitializing an active tick array deterministically fails with `ConstraintSpace`, breaking composed transactions and forcing clients to query and conditionally omit the public initializer.

Example Scenario:

1. A client initializes an empty non-liquidity-mining tick array with 64 bytes.
2. A position adds one tick, expanding the array to 240 bytes.
3. The client includes `initialize_tick_array` again as an idempotent prerequisite.
4. Anchor compares the declared 64-byte space with the actual 240-byte length and rejects the transaction before the handler runs.

```
#[account(
 init_if_needed,
 space = 8 + TickArray::INIT_SPACE
)]
pub tick_array: AccountLoader<'info, TickArray>,
```

Recommendations:

Replace the exact-space `init_if_needed` constraint with manual account initialization and accept existing arrays only after validating their PDA, owner, discriminator, header, and dynamic layout.

Alternatively, update the client SDK to check whether the tick-array PDA exists and only include `initialize_tick_array` when it does not. Use `getMultipleAccountsInfo` when checking multiple arrays.

Meteora: Resolved with PR-160.

Zenith: Verified.

[I-10] Incorrect and misspelled comments

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target:

- programs/dlmm-pro/src

Description:

Multiple comments in the original target commit do not match the code they describe, while others contain spelling or grammar errors. These issues may mislead readers and increase the cost of understanding and maintaining the codebase.

Semantic mismatches:

- lib.rs#L70 — `create_dynamic_config` is incorrectly labelled as creating a static config.
- ix_create_operator_account.rs#L36 — the comment says ten operations are supported, while `OperatorPermission` has eight variants.
- pool/mod.rs#L118 — the stated tick-ID range is only true when `tick_spacing = 1`; the represented tick IDs scale with tick spacing.
- pool/mod.rs#L125-L129 — the comment documents a remove-liquidity status bit that is not present in `PoolStatus`.
- pool/mod.rs#L596-L597 — `variable_fee_control` is a coefficient, not a basis-point value, so the stated `bps^2 × bps` dimensional explanation is inaccurate.
- base_fee_math.rs#L190 — the next bit is equal to `MAX_EXPONENTIAL`, not greater than it.
- tick_math.rs#L61 — `(sqrt_price^2 >> 64)` does not fit in `u128` for every value below `2^128`; the protocol range is safe, but the stated general bound is false.
- tick_math.rs#L319-L324 — the precomputed constants can exceed 105 bits and reachable intermediate products can exceed the claimed 210-bit bound.
- pool_reward.rs#L88 — `emission_per_second` is a `u128` fixed-point rate with 64 fractional bits, rather than a strictly fractional `Q0.64` value.
- pool_reward.rs#L95-L96 — the counter trAcknowledgeds zero active pool liquidity, not an empty tick, and these rewards are withdrawable as ineligible rewards rather than automatically carried into the next schedule.
- oracle.rs#L19-L22 — a tick-cumulative oracle is Uniswap V3-style; Uniswap V2 accumulates token prices instead of ticks.
- tick.rs#L182 — rounding the unfilled amount up favors the refundable unfilled side and reduces the calculated fulfilled amount, contrary to the comment.

Spelling and grammar errors:

- activation_time_based.rs#L44 — "bAcknowledged date" should be "bAcknowledgeddating."
- position_utils.rs#L183 and ix_withdraw_ineligible_rewards.rs#L96 — "short circut" should be "short circuit."
- tick_array_utils.rs#L252, ix_expand_position.rs#L66, and tick.rs#L456-L479 — "firstly" should be "first."
- ix_open_limit_order.rs#L310 — "limit orer" should be "limit order."
- ix_update_reward_authority.rs#L10 — "any one" should be "anyone."
- ix_claim_protocol_fee.rs#L133 — "shouln't" should be "shouldn't."
- ix_increase_oracle_length.rs#L32-L35 — "would need set" and "dont" should be "would need to set" and "don't."
- ix_swap.rs#L65, ix_swap.rs#L108, and ix_swap.rs#L414 — "dont," "trasfer," and "this happen" should be "don't," "transfer," and "this happens."
- lib.rs#L293 — "FOP" should be "FOR."
- base_fee_math.rs#L102 — "last 2 1's bit fulfill" should be "last two set bits fulfill."
- limit_order.rs#L42 and position_array/mod.rs#L55 — "should be not overflow" should be "should not overflow."
- pool/mod.rs#L144 — "initalized" should be "initialized."
- pool/mod.rs#L279 — "price is change" should be "price changes."
- tick.rs#L9 — "caculation" should be "calculation."
- tick.rs#L140 — "full-filled by increase" should be "fully filled by increasing."
- tick.rs#L755-L757 — "dont" should be "don't."
- tick_array_bitmap_extension.rs#L176 — "possitive_bitmap" should be "positive_bitmap."
- remaining_accounts_util.rs#L38 — "by consume" should be "by consuming."

Recommendations:

Correct or remove the listed comments so that each statement matches the current implementation and terminology.

Meteora:

Zenith:

[I-11] Q64.64 exponential helper uses non-exact reciprocal and incomplete boundary semantics

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target:

- base_fee_math.rs
- base_fee_math.rs
- time_scheduler_base_fee.rs
- market_cap_scheduler_base_fee.rs

Description:

The `pow` helper performs binary exponentiation in Q64.64 fixed-point arithmetic, truncating after every multiplication. Its reciprocal path uses `u128::MAX` (`2^128 - 1`) as the scaling numerator:

```
if squared_base ≥ result {
 squared_base = u128::MAX.checked_div(squared_base)?;
 invert = !invert;
}

// ... repeated Q64.64 multiplications ...

if invert {
 result = u128::MAX.checked_div(result)?;
}
```

For a Q64.64 value `x`, the reciprocal should be computed using the exact scale factor `2^128 / x`. Using `2^128 - 1` introduces a one-or-more raw-unit bias around unity and makes reciprocal identities such as `1^1 = 1` and `1^-1 = 1` inexact. The helper also returns `None` when the calculated result is zero:

```
if result == 0 {
 return None;
}
```

Consequently, `pow(0, positive_exponent)` is rejected even though it is mathematically zero, while `pow(0, negative_exponent)` is the genuinely undefined case. In addition, the repeated `>> 64` operations and the initial conversion of `reduction_factor` to Q64.64 systematically round toward zero, so the returned fee is an approximation of the real exponential curve rather than an exactly rounded result.

The helper is called by the exponential time and market-cap fee schedulers. Under the current validation rules, normal configurations use `0 < reduction_factor < 10_000`, which gives `0 < base < Q64`, and scheduler periods are non-negative `u16` values. Therefore the reciprocal branch and zero-base case are not reachable through a valid production configuration. There is currently no production impact through the existing execution paths, but potential issues may be introduced through future execution paths or reuse of the helper.

Recommendations:

Define and test the boundary semantics explicitly (`pow(Q64, n) = Q64`, `pow(0, n > 0) = 0`, and `pow(0, n < 0) = None`). Use the exact fixed-point reciprocal scale (`2^128`) with a sufficiently wide intermediate (for example, `U256`) and document the rounding direction. If exact fee reproducibility is required, keep the higher internal precision until the final conversion to the fee numerator.

Meteora: The relevant behavior was documented in @b4801db97d...

Zenith: Verified.

[I-12] Writable pool accounts unnecessarily serialize otherwise-independent claims

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target:

- ix_claim_reward.rs#11
- ix_claim_fee.rs#11

Description:

`ClaimRewardCtx` and `ClaimFeeCtx` provide pool configuration and metadata required to validate reward and fee claims.

However, both instructions mark the pool writable even though their handlers only read pool state. This takes an unnecessary write lock on the pool and prevents the claims from running concurrently with otherwise-independent transactions that also only read the pool.

Example Scenario

1. Two positions in one pool have claimable rewards for different reward indices bAcknowledgeded by distinct reward vaults.
2. Their owners submit the claims concurrently.
3. The positions, reward vaults, and destination token accounts are distinct, and both handlers only read the shared pool.
4. The unnecessary writable pool declaration serializes the transactions.
5. Marking the pool read-only removes that conflict; the two transactions can run concurrently provided they do not share another writable account.

```
#[account(mut)]
pub pool: AccountLoader<'info, Pool>,

#[account(
 mut,
 has_one = reserve_x,
```

```
 has_one = reserve_y,
 has_one = token_x_mint,
 has_one = token_y_mint,
)]
pub pool: AccountLoader<'info, Pool>,
```

Recommendations:

Remove the mutable account constraint from `pool` in both claim contexts because every current access loads it read-only.

```
pub pool: AccountLoader<'info, Pool>,

#[account(
 has_one = reserve_x,
 has_one = reserve_y,
 has_one = token_x_mint,
 has_one = token_y_mint,
)]
pub pool: AccountLoader<'info, Pool>,
```

Meteora: Resolved with @f858ad269d...

Zenith: Verified.

[I-13] Generic tick-array insertion bypasses the TickOps reference predicate

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target:

- tick_array_utils.rs
- mod.rs
- tick_reward.rs

Description:

`modify_tick_array_if_needed( ... )` is generic over `T: TickOps`, so its reference checks should use the trait's reference predicate. The function currently checks only the embedded base `Tick`:

```
let already_exists = ticks[..current_length]
 .iter()
 .any(|t| t.base().tick_id == id && !t.base().is_no_references());
```

This is inconsistent with the nearby `get_unique_tick_count( ... )` and `compact_ticks( ... )` helpers, which use the trait method:

```
t.is_no_references()
```

In the current implementation, `Tick` and `TickWithReward` are equivalent for this predicate because `TickWithReward::is_no_references( ... )` delegates to `self.inner.is_no_references()`. Therefore, this is not currently exploitable.

However, the function is intentionally generic. If a future tick extension stores additional reference-like state outside the base `Tick`, `modify_tick_array_if_needed( ... )` could treat such a tick as empty and reinitialize or duplicate state while other generic helpers would still treat it as referenced.

Recommendations:

Use the `TickOps` predicate consistently:

```
let already_exists = ticks[..current_length]
 .iter()
 .any(|t| t.base().tick_id == id && !t.is_no_references());
```

This keeps the generic helper aligned with the trait contract and avoids extension-specific reference state being ignored in future tick types.

Meteora: Resolved with @8fa7df090a...

Zenith: Verified.

[I-14] Several validation paths return misleading existing PoolError variants

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target:

- ix_add_liquidity_imbalance.rs
- ix_claim_reward.rs
- ix_swap.rs
- ix_initialize_reward.rs
- base_fee/mod.rs
- ix_set_pool_status.rs
- ix_add_liquidity.rs
- ix_remove_liquidity.rs
- activation/mod.rs
- ix_initialize_pool_with_dynamic_config.rs
- ix_update_reward_authority.rs

Description:

Several validation branches return an existing `PoolError` variant that does not match the actual failed condition. This does not directly bypass authorization or move funds incorrectly, but it weakens the protocol's error contract.

SDKs, frontends, indexers, monitoring systems, and integration tests often branch on specific errors. For example, slippage failures, zero-amount failures, invalid reward indices, invalid reward state, and malformed fee or activation configuration are different recovery cases. Returning a broad or unrelated error can make clients display the wrong reason, retry the wrong operation, misclassify production failures, or miss a real configuration issue.

The following checks should use already-defined errors:

- In `ix_add_liquidity_imbalance.rs`, the `tick_delta ⩽ max_tick_slippage` check currently returns `InvalidParameters`. This is a slippage guard and should return `ExceededSlippage`, matching the meaning used by other liquidity paths.
- In `ix_claim_reward.rs`, the `reward_index < NUM_REWARDS` bounds check currently returns `InvalidParameters`. This should return `InvalidRewardIndex`, consistent with the other reward instructions.

- In `ix_swap.rs`, the `input_amount > 0` check currently returns `InvalidParameters`. This should return `AmountIsZero`, since the rejected condition is specifically a zero swap input.
- In `ix_initialize_reward.rs`, an already-initialized reward slot currently returns `InvalidRewardIndex`. The index is valid; the reward slot state is invalid for initialization. This should return `InvalidRewardState`.
- In `base_fee/mod.rs`, decoding `base_fee.base_fee_mode` currently propagates `TypeCastFailed`. Because `InvalidBaseFeeMode` already exists, both base-fee mode decoding sites should map invalid base fee modes to `InvalidBaseFeeMode`.
- In `ix_set_pool_status.rs`, a `status` bitmask outside the allowed range currently returns `InvalidPermission`. The signer may have permission, but the supplied status value is invalid. This should return `InvalidParameters`.
- In `ix_add_liquidity.rs` and `ix_remove_liquidity.rs`, zero liquidity entries currently return `InvalidParameters`. These should return `LiquidityIsZero`, which already represents this condition more precisely.
- In `activation/mod.rs` and `ix_initialize_pool_with_dynamic_config.rs`, invalid activation-mode decoding currently propagates `TypeCastFailed`. Under the current error set, this should be mapped to `InvalidActivationAccess` rather than exposing a generic cast failure.
- In `ix_update_reward_authority.rs`, setting the reward authority to the current authority currently returns `IdenticalFunder`. This is not a funder comparison. Without adding a new error, `InvalidParameters` is the less misleading existing error.

These mismatches are mostly operational and integration risks rather than direct economic exploits. The main issue is that the program already defines more specific error variants, but some call sites do not use them consistently.

Recommendations:

Replace the mismatched `PoolError` values with the closest existing variants.

Meteora: Resolved with @c4c8e9c7a6....

Zenith: Verified.

[I-15] Protocol fee claims can fail for Token-2022 accounts requiring incoming transfer memos

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target:

- ix_claim_protocol_fee.rs
- token.rs

Description:

The `claim_protocol_fee2( ... )` instruction allows the protocol fee authority to withdraw accumulated protocol fees from either pool reserve to a receiver token account.

For pool-to-user transfers, the shared `transfer_from_pool( ... )` helper can optionally emit a Memo CPI before the token transfer when the destination Token-2022 account has the `MemoTransfer` extension enabled and requires incoming transfer memos. Most outbound user-facing flows pass a `MemoTransferContext` into this helper before transferring tokens.

However, `handle_claim_protocol_fee( ... )` calls `transfer_from_pool( ... )` with `None` as the memo context:

```
transfer_from_pool(
 &ctx.accounts.pool_authority.to_account_info(),
 token_mint,
 &token_vault.to_account_info(),
 &ctx.accounts.receiver_token_account.to_account_info(),
 &token_program.to_account_info(),
 amount,
 None,
 transfer_hook_accounts,
)?;
```

As a result, if the protocol fee receiver is a Token-2022 token account with `MemoTransfer.require_incoming_transfer_memos` enabled, the token program will reject the transfer because no memo was emitted immediately before the transfer. The protocol fee claim transaction will fail and the protocol fee authority will be unable to claim fees to that receiver account.

Recommendations:

Add memo support to `ClaimProtocolFee` in the same way as the other outbound transfer flows. The instruction should include the Memo program account, define a protocol-fee memo value, construct a `MemoTransferContext`, and pass `Some(memo_transfer_context)` into `transfer_from_pool( ... )`.

Meteora: Acknowledged, we dont use memo transfer in protocol fee.

Zenith: Acknowledged by client.

[I-16] Unreachable schedule endpoints in `lock_vesting_position` can permanently lock liquidity

SEVERITY: Informational	IMPACT: Informational
STATUS: Resolved	LIKELIHOOD: Low

Target:

- ix_lock_vesting.rs#18

Description:

`LockVestingPositionParameters::validate` checks that a vesting schedule begins no earlier than the current point and releases exactly `10,000` basis points.

However, it does not verify that `cliff_point + period_frequency * number_of_period` fits in the point domain.

The instruction can accept an irreversible finite schedule whose final unlock point can never be reached, permanently locking the selected liquidity.

Example Scenario:

1. A user sets the cliff to `u64::MAX - 5`, the period frequency to `10`, one period, and `10,000` basis points per period.
2. The parameters pass validation because the release basis points total `10,000`.
3. At the maximum possible point, only `5` points have elapsed, so integer division reports `0` elapsed periods.
4. The conceptual end point overflows `u64`, leaving all selected liquidity locked with no schedule replacement or cancellation path.

```
let total_release_bps = self
 .cliff_unlock_bps
 .safe_add(self.bps_per_period.safe_mul(self.number_of_period)?)?;
require!(total_release_bps == MAX_BASIS_POINT,
 PoolError::InvalidParameters);
```

Recommendations:

Reject schedules whose full duration or end point overflows and enforce any stricter domain bound required by the pool's clock source.

```
let duration = self
 .period_frequency
 .safe_mul(u64::from(self.number_of_period))?;
cliff_point.safe_add(duration)?;
```

Meteora: Resolved with PR-120 and PR-166.

Zenith: Verified.

[I-17] Partial limit-order fills in `swap` can strand uncredited input

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target:

- tick.rs#550

Description:

`calculate_filled_limit_order_amount_exact_in` converts exact input into a partial limit-order fill at the tick price.

However, the partial path rounds output down while marking the entire input as consumed, and maker proceeds are later reconstructed from output with downward rounding.

The taker can lose a large input remainder that is deposited into the reserve but is not credited to makers, fees, or unconsumed input.

Example Scenario:

1. One output base unit costs `1,000,000` input base units.
2. A partial fill supplies `1,999,999` input units and rounds output down to `1` unit.
3. The branch records all `1,999,999` units as consumed, while maker proceeds reconstructed from one output unit are approximately `1,000,000` units.
4. Approximately `999,999` input units remain in the reserve without an accounted claimant.

```
let out_amount = Tick::get_amount_out(amount_in, self.sqrt_price,
 swap_for_y)?;
Ok(LimitOrderSwapAmountFromInput {
 amount_in,
 amount_left: 0,
 out_amount,
})
```

Recommendations:

Recompute the input required for the rounded output with upward rounding and return the residual as `amount_left`.

```
let actual_amount_in = Tick::get_amount_in(
 self.sqrt_price,
 out_amount,
 swap_for_y,
 Rounding::Up,
)?;
```

Meteora: Acknowledged, 1 unit precision is acceptable in almost system.

Zenith: Acknowledged by client.

[I-18] `withdraw_ineligible_rewards` allows the reward authority to redirect unused campaign tokens

SEVERITY: Informational	IMPACT: Informational
STATUS: Acknowledged	LIKELIHOOD: Low

Target:

- ix_withdraw_ineligible_rewards.rs#53

Description:

`withdraw_ineligible_rewards` returns the unused portion of a completed reward campaign.

However, campaign state does not record the funding wallet or refund beneficiary, and withdrawal authenticates only the mutable reward authority while accepting any signer and destination token account.

The reward authority, including a newly assigned authority, can redirect tokens funded by another wallet to an arbitrary compatible destination.

Example Scenario:

1. Alice funds a campaign with `1,000` tokens.
2. A zero-liquidity interval leaves `300` tokens refundable.
3. The pool creator or operator changes the reward authority to Bob, or Bob is already the authority.
4. Bob supplies his signer and token account and withdraws the `300` unused tokens funded by Alice.

```
require!(
 reward_info.is_valid_authority(&self.authority.key()),
 PoolError::InvalidPermission
);
```

Recommendations:

Persist an immutable campaign refund recipient when funds are accepted and require both the signer and destination token-account owner to match it during withdrawal.

```
require_keys_eq!(reward_info.refund_recipient, self.funder.key(),
 PoolError::InvalidFunder);
require_keys_eq!(self.funder_token_account.owner, self.funder.key(),
 PoolError::InvalidFunderTokenAccount);
```

Meteora: Acknowledged, added a fix in attached commit @d0de080835....

Zenith: Acknowledged by client.